

Building static robots to solve
manipulative puzzles
Ranulf Green
BSc Computer Science
Session 2006/2007

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of student) _____

Summary

Numerous manipulative puzzle problems have been invented all requiring interesting and sometimes complex strategies to solve them. Memory and reactions are required to minimise the time taken to solve the puzzle even if the strategy used was not invented by the person solving the puzzle. Can a robot solve a manipulative puzzle faster than a human? Can a robot take advantage of a computer's superior mathematical processing speed to increase the efficiency of the strategy used further than is possible for a person?

This project aims to answer these questions for one particular puzzle problem highlighting the difficulties of developing solutions to each aspect of the problem. The chosen puzzle is analysed in detail and two strategies are developed for finding a solution. Solutions are then analysed against robotic performance to assess which is better. Finally the system is compared with the performance of a person on the chosen puzzle problem.

A video of a test solution, links to useful related websites, source code and the software that has been implemented can be found at:

<http://www.comp.leeds.ac.uk/qsr/rubik>

Acknowledgements

Firstly, I would like to thank Tony Cohn for his continued support, advice and interest throughout the project. Secondly, I would like to thank Haiko Muller for his feedback on my mid-project report and project demonstration. Thirdly, I would like to thank the School of Computing for the supply of Lego Mindstorms equipment without which, this project may not have been possible. Finally I would like to thank all those who expressed interest in my project providing me with the motivations to continue development even in the face of failure.

Contents

Summary.....	I
Acknowledgements	II
Contents	III
Chapter 1 Introduction.....	1
1.1 Aim.....	1
1.2 Objectives.....	1
1.3 Minimum requirement, constraints and deliverables	2
1.4 Relevance to Degree program	2
1.5 Project Layout	2
Chapter 2 Problem background.....	3
2.1 Introduction	3
2.2 Chosen puzzle problem	3
2.3 The Rubik's cube.....	3
2.3.1 Orientation.....	4
2.3.2 Position	4
2.3.3 Lemma 1	5
2.3.4 Lemma 2.....	5
2.3.5 Summary.....	6
2.4 RCX:.....	6
2.4.1 Hardware	6
2.4.2 Firmware.....	6
2.5 Infrared Communications.....	7
2.6 Logical Model	8
2.7 Sensors	9
2.7.1 Rotation sensors.....	9
2.7.2 Timing	9
2.7.3 Touch sensors	10
2.7.4 Vision	10
2.8 Solution algorithm.....	10
2.8.1 Candidate algorithm 1: Seven stage algorithm.....	10
2.8.2 Candidate algorithm 2: Five stage algorithm.....	11
2.8.3 Candidate algorithm 3: Two phase algorithm	11
2.8.4 Informed search algorithms	13

2.9 Capturing the initial state	13
2.9.1 User input	13
2.9.2 Automatic input	13
Chapter 3 Development methodology and planning.....	14
3.1 Introduction	14
3.2 Waterfall model	14
3.3 Unified Software Development Process (USDP).....	15
3.4 Chosen development methodology	15
3.5 Project plan.....	17
3.5.1 Alteration 1: Extended time required to develop a basic system.....	17
3.5.2 Alteration 2: Refining the system.....	17
3.5.3 Alteration 3: Scrapping the vision system.....	17
Chapter 4 Solution	18
4.1 Introduction	18
4.2 General system design.....	18
4.3 Basic system	20
4.3.1 Introduction	20
4.3.2 Robot	20
4.3.3 RCX programs.....	21
4.3.4 Communications protocol.....	23
4.3.5 Move translation service.....	25
4.3.6 Logical cube	26
4.3.7 Solver.....	29
4.3.8 Interface	33
4.3.9 Combined testing.....	34
4.4 Improved system	35
4.4.1 Introduction	35
4.4.2 Improved logical model.....	35
4.4.3 Two Phase Solver	38
Chapter 5 Evaluation.....	43
5.1 Evaluation criteria	43
5.2 Minimum requirements	43
5.2.1 Justification of minimum requirements criterion	43
5.2.2 Evaluation against minimum requirements	43
5.3 Non-robotic solution speed	44
5.3.1 Justification of non-robotic solution speed criterion	44
5.3.2 Evaluation of non-robotic solution speed to solution speed of the system.....	45

5.3 Further project aims.....	46
5.3.1 Justifications of further project aims criterion.....	46
5.3.2 Evaluation against reduced number of moves in generated solution.....	46
5.4 Existing solutions	46
5.4.1 Justification of existing solutions criterion.....	46
5.4.2 Evaluation against solving software implementing the seven stage method.....	47
5.4.3 Evaluation against solving software implementing two phase method.....	47
5.5 Effectiveness of methodology	48
5.5.1 Justification for effectiveness of methodology criterion	48
5.5.2 Evaluation against effectiveness of methodology	48
Chapter 6 Conclusion and future work	49
6.1 Future work	49
6.1.1 Vision system	49
6.1.2 Advancements in strategy to find a solution.....	49
6.2 Conclusions	50
Bibliography	51
Appendix A Personal reflections	54
Appendix B Test results.....	56
Appendix C Initial Plan.....	58
Appendix D Definitions	59

Chapter 1

Introduction

1.1 Aim

A person may develop a strategy to solve a manipulative puzzle such as the Rubik's cube [1], Towers of Hanoi or the sliding tile problem. Manipulation of such problems generally suits the complexities of the human hand and arm motion rather than the limited movements available to a simple robot, for here the person has a large advantage over the robot. However, human memory and arithmetical ability limits the complexity of the strategy and thus the speed of the solution. Can a robot be used to improve upon such limitations? In this project a robot is developed to solve a chosen example of a manipulative puzzle. The report analyzes the process of manipulating the puzzle, from capturing the puzzles details to carrying out the robotic movements required to manipulate the puzzle to its solved state.

1.2 Objectives

- Choose a suitable example of a manipulative puzzle. This is defined by the complexity of each movement. If a movement requirement is too complex then the puzzle might be out of the capabilities of the project.
- To construct a static robot with the physical characteristics required to perform the actions required to solve the chosen puzzle problem. The robot must have the functionality to perform every movement required to solve the puzzle with minimal human interaction.
- To create an algorithm which can generate a list of manipulations such that when these manipulations are applied to the scrambled state, the solved state is generated.
- To choose an appropriate software platform to implement the programmable robot. This includes the firmware that runs the robot as well as the programming environment for the base computer.
- To implement the software enabling the robot to manipulate the puzzle to the solved state. The robot must be able to use all its controls to a suitable degree of accuracy.
- To research and implement further enhancements to the system to take advantage of computer speed and memory thus minimising the time the robot requires to solve the puzzle.
 - Vision component to track errors and gather the puzzles starting state
 - Enhanced software model and algorithm to generate the solution

1.3 Minimum requirement, constraints and deliverables

Due to the physical nature of the project, there is a certain degree of inaccuracy involved in the manipulation of the chosen puzzle problem. The minimum requirement is to construct a programmed robot that fulfils the first five objectives such that the implemented robot is able to manipulate at least one non-solved input puzzle to the solved position. In the minimum case the program will not be required to automatically collect information about the input puzzle. The project is limited by two main constraints: Firstly, the amount of time available, as defined in the schedule, restricts the potential complexity of the system hence limiting the development of advancements noted in 1.2 point 6. The second constraint is the development platform. A bespoke robot, with high levels of efficiency and power may require experience in both electronics and engineering, both of which are not relevant to a computer science degree. There are 3 main project deliverables: A robot with the physical capabilities to solve the chosen puzzle; Software to control the robot such that the puzzle may be solved and the project report. Further to the main deliverables, a URL pointing to a website containing the implemented system, all external required software and instructions for use will be provided.

1.4 Relevance to Degree program

The Developer's degree program is BSc in Computer Science (CS). Software engineering, programming fundamentals, architecture and organisation of computer systems and algorithm comprise core areas of computer science [24]. This project aims to draw on all of these aspects. Understanding the mathematical basis of the problem and designing an algorithm to solve the problem focuses on the skills covered in theoretical computer science. Implementing a full scale solution draws on the Developer's knowledge of computer architecture, programming languages and methodologies to develop software. The content is aimed such that developing a solution is pivotal upon the skills the Developer has learned throughout a CS degree. Often design decisions will be made based upon the skills the Developer has gathered during a CS degree. The main influence here is the bias towards programming in Java [9] thus developing using the object orientated paradigm.

1.5 Project Layout

Chapter 2 includes details of the research, the Developer carried out, used to aid development of the system. Chapter 3 includes an evaluation of two commonly used methodologies, an explanation of the chosen methodology and details of the project plan. Chapter 4 includes the design, implementation and testing involved in the construction of a basic and improved system. Chapter 5 contains an evaluation, using several important criteria compared with the relevant system components, of the successes and/or failures of the project. Chapter 6 covers suggested future projects together with basic details of how these projects may be carried out.

Chapter 2

Problem background

2.1 Introduction

This chapter outlines the investigation that the developer carried out in order to design a system to satisfy the aims stated in section 1.2. The chapter is split into important sub-domains of the entire problem that encompass the important information that the Developer has learned.

2.2 Chosen puzzle problem

The chosen puzzle problem is the Rubik's cube. The motivations for this choice are simple: The Rubik's cube presents an interesting and complex mathematical problem that cannot simply be solved by brute force. Therefore ingenuity and research will be required to produce a solution. Justifications for these claims are outlined in section 2.3.

2.3 The Rubik's cube

The Rubik's cube, often referred to as the 'cube', is a regular hexahedron made up from 26 separate smaller cubes or 'sub-cubes' consisting of 8 'corner' sub-cubes each with 3 coloured faces or 'sub-faces' and 3 possible orientations, 12 'edge' sub-cubes each with 2 coloured faces and 2 possible orientations and 6 'centre' cubes each with one coloured face and 1 possible orientation. The letter notation [2] for representing cube sides is the most commonly used notation and so hence forth will be used. Figure 2.1 illustrates a corner sub-cube, an edge sub cube and the faces U, F, R (the other faces are hidden).

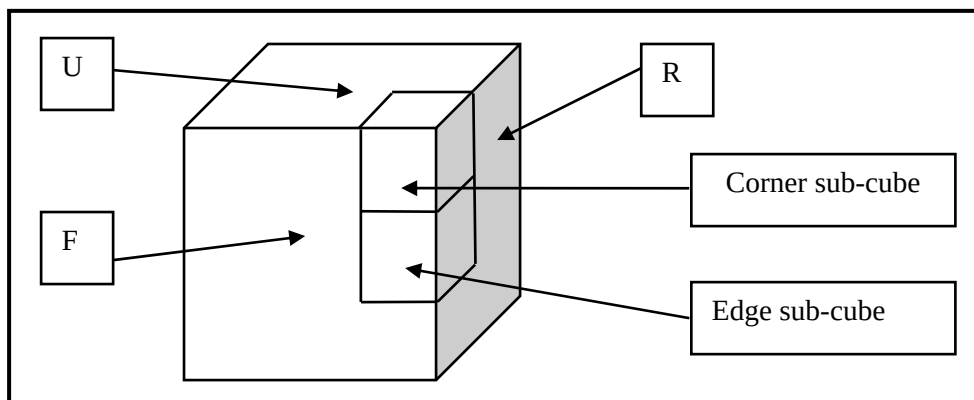


Figure 2.1: The Rubik's cube

The 18 possible cube manipulations are made up by twisting any of the faces in one of three possible ways: clockwise 90° , anticlockwise 90° or 180° . A face rotation will be denoted by one of the terms: F, B, U, D, L, R, FF, BB, UU, DD, LL, RR, F2, B2, U2, D2, L2 or R2. The letter stands for the face in which to turn (Front, Back, Up, Down, Left, Right). Double letters represent anticlockwise turns and 180 degree turns are followed by a '2'. A re-arrangement of the sub-cubes is called a 'permutation' of the cube [2, 12]. This rearrangement may be carried out through performing a sequence of face rotations or a 'move list'. The problem domain for the Rubik's cube is the set of all permutations of the cube that are reachable by applying any combination of the above face rotations. Each sub-cube (with the exception of the centre piece sub-cubes) has 2 attributes:

2.3.1 Orientation

There are many ways to define orientation however in [3] a simple and effective method defining 'sub-cube face references' is used to determine orientation. The orientations of the corners and edges are determined by their positions relative to the reference faces. Orientations are changed according to the face rotations that are applied to the permutation (see appendix D for definitions). When applying one face rotation after another we simply add the change in orientation for each move individually. For example, we track the Front up edge: FF moves the front up edge to the front left edge and flips its orientation. LL move the front left edge to the 'left up' edge but orientation does not change since only F, FF, B or BB affect edge orientation. If we then apply UU the edge is returned to its original position but in a different orientation. There are 4478976 permutations of orientation. By Lemma 1 the only 7 of 8 edges can be independently orientated.

- **3^7 Corner orientations**
- **2^{11} Edge orientations**

2.3.2 Position

Sub-cubes can be moved around the cube by applying face rotations (see appendix D for definitions). If the cube were to be dismantled then put back together in a random order the resulting permutation would be one of 1.92×10^{13} possibilities, only taking sub-cube position into account.

- **12! Edges positions**
- **8! Corner positions**

2.3.3 Lemma 1

“There exists no combination of moves that will re-orientate exactly 1 edge or exactly 1 corner.”

Proof

The definitions for orientation changes (see appendix D for full list) are as follows: Applying any one of the rotations L, LL, R or RR: 0 edges are flipped, 2 corners are twisted anticlockwise and 2 corners twisted clockwise. By applying any one of the face rotations F, FF, B or BB: 4 edges are flipped, 2 corners are twisted clockwise and 2 corners twisted anticlockwise. A combination of the other moves may change the sub-cubes that are re-orientated in any of the 4 rotations above. A move list M is a set of face rotations of length N that, once applied to the input permutation P_a , result in a new permutation P_b . To prove Lemma 1 we induct on the size of M which contains N face rotations such that if M^* which contains N+1 face rotations holds Lemma 1 then Lemma 1 holds for all $N \geq 0$.

Base case

Consider the empty move list M and the permutation P_a such that Lemma 1 holds for P_a . Then since no face rotations are carried out the resulting permutation $P_b = P_a$ hence Lemma 1 holds for $N=0$.

Induction

Consider any move list M containing N face rotations where $N > 0$. M is applied to the solved state and the resulting permutation is P_a such that P_a holds Lemma 1. Then by applying any face rotation to P_a the resulting permutation P_b must contain: either 0 or 4 flipped edges; either 0 clockwise and 0 anticlockwise twisted corners or 2 clockwise and 2 anticlockwise twisted corners hence since M does not re-orientate any single edge or corner, move list M^* containing N+1 face rotations does not re-orientate exactly one edge or exactly one corner thus Lemma 1 holds for M^*

2.3.4 Lemma 2

“The total permutation of corners and edges must be even”

The movements of sub-cubes during face rotations can be represented in terms of cycles of sub-cubes. For example, a 2-cycle resembles the swapping of 2 sub-cubes and a 4-cycle resembles the rotating of a 4 sub-cubes chain. If a face rotation is applied then 4 edges and 4 corners are rotated as part of 2 4-cycles. These 4-cycles can be written as 3 consecutive transpositions. If we take the edges and the F face rotation as an example: The U edge swapped with the R edge, the new U edge (old R edge) is swapped with the D edge and finally the new U edge (old D edge) is swapped with the L edge. According to [2], since there have been 3 transpositions the new permutation of edges is odd. “On the

cube, any basic move is a product of 2 4-cycles, hence is even.” [2] Here two 4-cycles (each resulting in an odd permutation) equal one even cycle thus an even permutation. A combination of two face rotation contains 4 4-cycles hence a combination of N face rotations contains N 4-cycles hence is even hence Lemma 2 holds. Since odd permutations of corners and edge cubes are not possible then the swapping of just 2 edge sub-cubes or just 2 corner sub-cubes, a 2-cycle is not possible.

2.3.5 Summary

As we have seen, not all positions or orientations are achievable. By Lemma 1: if an edge or a corner is taken out from the cube and the orientation is changed then replaced the cube cannot be solved. However, one may orientate all the other corners or edges as they wish so the orientation of the last edge or the last corner is forced by the orientations of the other edges or corners. By Lemma 2 it is possible to see that the total number of possible permutations of corner and edge sub-cubes is half of the figure previously suggested since odd permutations of corner and edge sub-cubes are not reachable. Thus the total number of possible cube permutations is $(7! \cdot 11!) / 2 \cdot 2^{11} \cdot 3^7 = \sim 4.3 \cdot 10^{19}$. This large number ensures that simply storing solutions to all possible permutations would be out of the question. It is therefore necessary to split the problem down into steps that can be solved independently of one another. Further exploration of these steps is outlined in 2.8.

2.4 RCX:

2.4.1 Hardware

The RCX (Robotics Command Explorer) [4] is a simple computer based on a Hitachi 8-bit 16MHz microprocessor [23]. The RCX has infrared, speaker and LCD outputs along with the three motor output terminals. Input is taken from three sensor inputs, through any of the RCX buttons or through the IR port. The main limiting factor of the RCX is the small amount of memory that is available. Each RCX has 32Kb RAM to store programs and runtime variables.

2.4.2 Firmware

By default the RCX comes preinstalled with the Lego RCX Firmware. Programs can be written, uploaded and run using the Lego RIS software environment [4]. RIS provides a simple to use graphical method for constructing high level programs. In [5, 22, 23] it is suggested the simplicity of the RIS constrains the power of the RCX thus limiting its use in large, complicated systems. Further, the RIS does not provide functionality for the development of multi RCX communications as required for this project (see 2.5 for details). For these problems, using the default firmware in conjunction with the RIS development platform is unsuitable for this project.

LegOS and NQC

Since its release, continued attempts have been made to ‘hack’ the RCX byte code in order to extract the low level instructions required to run commands such as ‘Motor A forward’. One method that has been developed to enable this is NQC [6]. This is a simplified version of the C programming language that enables the programmer to construct and compile programs and run them using the original firmware. LegOS [7] is a more advanced system that completely replaces the RCX firmware to enable the full functionality without the limitations of the original processor. Whilst both these implementations offer advanced functionality, extra time would be required for the Developer to learn the C programming language.

LeJOS

LeJOS [8] is a java implemented firmware replacement. The advantages of LeJOS stem through the simplified approach to download programs and access to RCX functionality using java APIs. Since the Developer has expertise in Java, programming and the Lejos system sustains an active development community and the successful application of LeJOS in [26], using Lejos makes a sensible choice.

Garbage collection

Standard Java supports garbage collection enabling old unused variables to be removed from memory during runtime preventing the memory requirement of a program from expanding unnecessarily during execution. LeJOS has reduced functionality: standard Java garbage collection is not supported, thus each newly created variable will occupy RCX memory for the duration of execution. Combined with the RCX’s low availability of memory, RCX programs will need to be designed such that memory isn’t used up during the lengthy execution of the program. This can be done by initialising all variables at program start.

2.5 Infrared Communications

Low level messages may be passed between USB infrared tower and RCX infrared port. Since more than one RCX will be required to control all functionality, a communications protocol will need to be devised to send and receive messages to one of two RCX modules independently. The LeJOS ‘pcrcxcomm’ and ‘rcxcomm’ APIs [8] have a range of communications protocols available based upon the java communications API [9]. The API contains an ‘RCXLNPAddressingPort’ which uses an address to ensure that a message is received by the correct RCX. The LNP (Lego network protocol) ensures that messages are not corrupted but does not ensure that they are fully transmitted. Further, it is an advantage to use these pre-built protocols over a fully bespoke system since the USB tower is already fully integrated. I/O streams can be retrieved using the port’s ‘getInputStream’ and

‘getOutputStream’ methods and a sequence of bytes can be transmitted using the I/O streams ‘read’ and ‘write’ methods. This allows simple byte messages to be sent to a specific RCX dictating the next action or sensor reading. A limitation of the ‘RCXLNPAddressingPort’ is that an instance can only be used to communicate with only one RCX and a program can only have one instance running at any one time. To combat this problem a new port can be opened using the required address each time a message is to be sent from the host machine. The port can then be closed using the ‘close’ method, after the message has been sent, ready for the next port to be opened. A past project [25] illustrates, through testing, a 50ms delay is required from sending one message to the next to prevent the IR port from reading a reflection of the first message and a 200ms delay before sending a response “to ensure that the RCX that sent the original message is ready to receive the reply” [25].

2.6 Logical Model

To solve a permutation of the cube a logical model representing the cube is required such that face rotations can be applied and the results stored and analysed without having to manipulate the actual cube. A simple method to do this is to store two dimensional arrays representing each face of the cube containing values between 1 and 6 that represent the colour of each sub-face (sub-cube face). The array values can be updated accordingly after a face rotation is carried out. “cube explorer” [3] implements a more complicated method applying two further abstractions of logical model to improve the speed to calculate the results of applying a face rotation: Firstly the ‘sub-cube’ approach models permutations as sub-cubes rather than sub-faces by storing each sub-cubes position and orientation in two ordered lists: for corners and edges. The second abstraction uses coordinates to distinctly represent each permutation of the cube. The sub-face method requires 20 table updates to be made for every move (8 updates for the face in question and 12 updates for the edge and corner faces of all the connected faces). The sub-cube method requires only 8 table updates for position and a maximum of 8 updates for the orientations. As described in [10], ‘move-tables’ contain the resulting coordinates from face rotations for each possible input coordinate, hence only one move table lookup for each coordinate is required for each face rotation.

Magnitude		1	2	3	4	5	6	7
Corner	URF	ULF	ULB	URB	DRF	DLF	DLB	DRB
Is replaced by	ULF	DLF	ULB	URB	URF	DRF	DLB	DRB
Value stored		0	1	1	4	1	0	0
Coordinate	$(7! * 0) + (6! * 0) + (5! * 1) + (4! * 4) + (3! * 1) + (2! * 1) + (1! * 0) = 224$							

Figure 2.2: The required positional changes after the move ‘F’

Figure 2.2 illustrates an example of the encoding scheme for corner positions as used in [3] (URF: Up right front, ULF: Up left front etc.) For each magnitude, starting with the largest, multiply the factorial of that magnitude by the number of remaining sub-cubes with higher magnitude than the actual sub-cube residing at the position in question then remove this sub-cube from the list. The sum of these values is the coordinate. No value is stored for the sub-cube at URF since the sub-cube here is forced by the preceding 7 corners. Implementing a system with coordinates would require generation of stored move tables for each coordinate. If the cost of generating the move tables outweighs the performance benefits of using a coordinate based system then the tables can be generated prior to execution of the solution and stored in persistent memory. Co-ordinate based systems would be useful when the cost of generating a single move is a defining factor in solving the cube. In [3] persistent move tables are generated on first execution of the program, later to be loaded back into memory, to reduce the time required to generate each solution.

2.7 Sensors

When a robot carries out a face rotation some perception of the movement is required to ensure that the correct degree of rotation is accomplished. Below are four candidate solutions:

2.7.1 Rotation sensors

A rotation sensor can be used to record the movement of the motor that manipulates the robot. In [26] a rotational sensor is used to calculate the angle the robot has turned. A robot jaw, such as that required to grip a Rubik's cube, can make use of a rotation sensor to control jaw movement by monitoring the movement of the motor controlling the jaw movement. As found in [18], using a simple experiment, it was shown that Lego parts may flex when put under pressure. Flex may cause a variation between the actual movement of the jaw and the measured movement of the jaw. If this variation is more than the required precision for that movement then rotational sensors will not be suitable for the application.

2.7.2 Timing

No Sensors are required to simply turn a motor on for a set amount of time making timing a useful method when no RCX sensor ports are available. By monitoring the amount of rotation for two motors over a set amount of time, it was seen that motors may not run at the same speed. This affect was considerably increased when one of the motors is placed under resistance thus limiting the use of timing in situations where the force opposing the motor may vary. Timing may still be used on top of sensor readings to sense if a process is jammed or unable to fully complete the required movement.

2.7.3 Touch sensors

Touch sensors are generally used to detect contact with an object [22]. In [26] touch sensors are used to detect contact with the walls of a maze. In the context of this project touch sensors can be used to measure jaw rotation if the resistance, caused by rotating a face of the Rubik's cube, causes the system to flex thus creating a variation between motor rotation and jaw movement.

2.7.4 Vision

Using a video camera, the sub-faces of the Rubik's cube may be detected using a process such as the widely used 'canny edge detector' [27]. When conducting a face rotation, the system can halt the rotation based on the alignment of the detected edges of the cube. Since edges in images represent areas with high levels of variation between pixel values [27], if these levels are decreased, perhaps through changeable lighting conditions, then the camera will need to be adjusted to compensate. Since touch and rotation sensors are easier to implement it does not make sense to use vision to detect cube movements; however, a vision system may be used in conjunction with the sensors to verify the accuracy of a movement.

2.8 Solution algorithm

For each increase in move sequence length the number of possible combinations would increase by a factor of 18 until every possible permutation is reached. In [2] the time taken to count all these possibilities is estimated at 1.4 million years. To combat these problems it is necessary to split the solution into sections such that each section can be completed independently from the previous section. This approach enables only features of the permutation required for a particular section to be regarded in the early stages and decreases the number of possible permutations in the late stages.

2.8.1 Candidate algorithm 1: Seven stage algorithm

This is one of the oldest approaches to solve the Rubik's cube. Each step is stated in [2] although the method appears in many sources. The approach is to solve the cube in the following 7 steps:

- Place the top layer edge pieces in the correct orientation and position.
- Place the top layer corners in the correct orientation and position.
- Place the second layer edge pieces such that the top two layers are solved.
- Orientate (but don't place) the bottom layer edges.
- Place (but don't orientate) the bottom layer corners.
- Orientate the entire bottom layer
- Place the bottom layer edge pieces without effecting the orientations.

The problem is split such that the number of possible permutations for relevant sub-cubes in each stage is vastly reduced. The first stage can be solved by trial and error and the subsequent stages can be solved by applying varying move lists depending on the permutation of the relevant sub-cubes for that stage. Having seven stages limits this number of permutations such that very few move lists are required to complete each stage. Progress is clearly visible throughout reconstruction of the cube making the seven stage algorithm a simple to learn approach for beginner ‘cubists’. However, with simplicity comes in-efficiency due to the number of stages since extra moves have to be carried out to ensure that between each stage the cube returns to its semi-solved state. For example, if a particular sub-cube X needs to be permuted from place A to place B, a face rotation that does such a transposition may also affect the position of sub-cube Y. If Y is in its required position and orientation already then after X has been moved, Y will have to be moved back to its starting place and orientation resulting in one or more extra face rotations on top of those required to place X. Applying the 7 stage algorithm manually several times resulted in solutions generally over 100 face rotations.

2.8.2 Candidate algorithm 2: Five stage algorithm

Dan Harris’ and Lars Vandenberg have invented the VH algorithm [11] that uses 5 stages instead of 7 to reduce the number of extra moves required by solving elements from more than one layer at the same time.

- Place the top layer edge pieces in the correct position and orientation.
- Place and orientate the first three top layer corner and second layer edge pieces
- Place and orientate the final top layer corner, second layer edge combination and orientate the bottom layer edge pieces.
- Place and orientate the bottom layer corners
- Place the bottom layer edges

As can be imagined, due to the decrease in the number of stages, there is an increase in the number of possible permutations for each stage thus a larger number of move lists need to be learnt to produce a solution. For example in stage 4 of [11], applying 1 of 32 move lists will result in a solution for that stage depending on the input permutation. The VH algorithm shows an increased efficiency over the seven stage algorithm. However, this comes with an increased complexity due to the large number of move lists required for each stage.

2.8.3 Candidate algorithm 3: Two phase algorithm

Kociemba’s 2 phase algorithm [3] is a refinement of Thistlethwaites 5 stage algorithm [2, 12]. Solutions found using [3] represents a significant improvement in performance over algorithm 1 and 2.

In [3] it is shown, by applying the two phase method to 1 million random input cubes, a maximum of 29 face rotations are required to find a solution. The idea is to divide the search space into two even subgroups. The first phase group contains all the possible permutations. The second phase group is a sub-group of the first phase, containing all permutations reachable by applying any combination of U, UU, D, DD, L2, R2, F2 or B2. These face rotations are chosen such that no orientation of any sub-cube are affected by applying any of the face rotations hence the first phase can be completed by, ignoring all the positions, solving the corner and edge orientations and the second phase, that results in the solved state, can be solved by applying only the above face rotations. To even out the size of the search space from phase 1 to phase 2 (orientations to positions as seen in 2.3.5), the system places all the middle layer edges in the middle layer in no particular order in phase 1. In [3, 10] it is stated that Michael Read proved the two phase algorithm can solve a random cube in 30 moves or less (at most 12 moves for phase 1 and 18 moves for phase 2). If the first phase solves all the coordinates then the solution is optimal hence, as demonstrated in [3], by trying sub-optimal results for the first phase the size of the second phase is reduced.

Cycling through combinations of face rotations of length up to 18 produces a huge number of possible move lists thus simply running through each possibility would take too long. The efficiency of the algorithm may be increased if the search is conducted in the correct direction using an informed search method. In [13] it is suggested solutions can be found using an informed search more efficiently than using an uninformed search. The informed search becomes useful for the purposes of creating a solving algorithm, which cycles large lengths of move lists. The heuristic function may take several forms. The number of misplaced or miss-oriented sub-cubes may be used or the distance of these sub-cubes to their goal positions, the city block distance [13], may be used. In [3] the heuristic function uses a ‘pruning table’ containing an index value for every permutation of the coordinates relevant to each phase. The pruning table is built by conducting a breadth first tree search, starting with the solved state, conducting each face rotation to each leaf at every level down to a pre-specified depth. At each vertex the pruning table value indexing the permutation of that vertex is then set to the number of face rotations used to reach the vertex. Since each stored value represents the minimum distance to the goal state, the pruning table provides an ‘admissible heuristic’ thus guaranteeing the optimality of the A* search algorithm [13, 28]. At any particular permutation, for each face rotation the resultant permutation can be looked up in the pruning table. The value returned represents the number of face rotations required to reach the goal state thus the next node to expand is the node closest to the goal state. The first phase pruning table in [3] uses an exact metric for the distance to the goal state since all the coordinates are used to index the pruning table. The second phase in [3] uses an estimate since certain coordinates are disregarded to reduce the size of the pruning table.

2.8.4 Informed search algorithms

The previously mentioned A* search algorithm expands nodes based on the estimated cost of a node to the goal state added to the actual cost of reaching that node. Since the pruning tables provide such an estimate, A* makes a good choice; however, A* is said to consume too much memory for practical use [13, 28] as generated nodes collect in memory. In [13] the IDA* algorithm is suggested for problems using unit step costs and hence is used in [3]. IDA* uses the process of iterative deepening enabling use of the depth first search for each depth rather than the “inordinate amount of space” [28] a breadth first search requires. In the context of the Rubik’s cube, as described in [28] the IDA* algorithm takes the following steps: Unless the goal state is already found, set the threshold to 1. Conduct a depth-first search removing nodes with pruning table indexes exceeding the threshold; if the solutions is found return, else set the threshold to the smallest number larger than the current threshold (lower bound estimate of the number of face rotations required) and repeat the search.

2.9 Capturing the initial state

2.9.1 User input

The simplest way to capture the cube’s initial state is to input each sub-face colour manually. Java’s Swing framework [9] provides a ‘Grid Layout’, ideal for the positioning of each of the elements in a network view of the cube. The colour to paint the elements may be defined by the colour selected in a ‘paint palette’, represented in the form of a series of coloured buttons which, when clicked, denote the colour in which to re-colour the network element. This method is simple to implement however requires extra input from the user.

2.9.2 Automatic input

A camera may be used to collect the initial state of the cube similar to that used in [3] and [18]. This system requires less user input than 2.9.1 and so represents a better solution than that of 2.9.1. The Lejos Vision API [8] provides an in built colour listener which is successfully implemented in [26] for colour detection. However, through testing ‘cube explorer’ [3] in varying lighting conditions from bright to dim, it becomes difficult to define the differences from white to yellow and yellow to orange in bright lighting conditions. The HSI colour model allows the intensity and the colour to be varied independently [27], allowing the saturation of white light in the bright images to be reduced thus resulting in a clearer definition between colours. If the brightness of the source light is measured then a suitable adjustment can be made. The camera signal can be split into regions using the Lejos Vision system and light measurements taken for each region thus accommodating for local variations in lighting conditions. Although a vision system does represent an improved solution to a user input system, the implementation will be much more complicated.

Chapter 3

Development methodology and planning

3.1 Introduction

The purpose of this project is to develop a robot that can solve manipulative puzzle problems, namely the Rubik's cube. The level to which this problem may be solved is variable, that is one solution may be superseded by another more efficient solution and it is difficult to gauge how advanced a solution is possible given the project constraints, hence flexibility is an important factor influencing the choice of methodology. The project is intended as a learning piece rather than a fully developed product thus little effort will be taken to deliver the project free of minor bugs particularly in the user interface, hence the methodology will contain a bias towards improved development over extensive testing. Below is a critical analysis of 2 commonly used approaches to software development in the context of this project:

3.2 Waterfall model

The waterfall model, as described in [14], defines a strict, inflexible process for the development of software. Due to the experimental nature of this project, such a strict process may not be ideal. If it becomes clear that the solving part of the system requires a more efficient logical model then the logical model will have to be redesigned. However, this is not possible since the design phase has already taken place. The stages defined in the waterfall model provide a 'basis' in which an iterative, fluid approach may be adopted. During the first stage: 'Systems engineering', a structure for the system is defined. Here general decisions can be made for the chosen platform for development and an outline for what needs to be done can be compiled. This process will be useful for the project to define the interaction between components allowing each component to be designed, constructed and tested independently thus reducing the complexity of the problem. The general system architecture may start as a general view of components then added to as the exact details of the interaction between components becomes clear. Some projects may suit the waterfall model methodology: "The process is best suited to projects where the requirements are not likely to change and where the requirements are fully understood" [15]. In the context of this project the requirements become steadily more complicated as new techniques are discovered and the Developer's capabilities expand so the waterfall model may not be a good basis for development. Due to these problems, the waterfall model is not a sensible choice for this project, but the model does provide a sensible guide to create a bespoke flexible methodology specific to experimental projects.

3.3 Unified Software Development Process (USDP)

By analysing the waterfall model, it became clear that a flexible process is required, allowing the problem to be split into achievable components. The USDP [14, 16] is a modern approach to software development based on underlying principles rather than strict stages thus allowing the project Manager to incorporate system specific influences into the process. In [14] the USDP stresses the importance of splitting the problem into components, developing the system from the requirements using techniques such as ‘use case realisation’ and to develop the system in phases marked by milestones. The system components can be mapped to the areas, covered in the research, thought to be important to the project. Since the scope of what is possible is not clear at this stage, using milestones to mark the end of important development lifecycles or phases presents a way of allowing functionality to be included without risking critical increases in the project’s complexity. Whilst many aspects of USDP suite this project, requirements driven development will create extra unnecessary work. Since the user input is simple, the final output is clear and much of the system complexity may be found in the control layer of the system. The USDP provides the flexibility required for this project and allows the problem to be broken into several phases and components thus reducing the chance of failure and difficulty of the problem. USDP also provides the Developer with a means to apply familiar object oriented coding practices. USDP provides a project Manager with the tools to control the developments of systems with many actors and complicated interaction between possibly widely distributed components developed by independent teams. This project is architecturally simple and managed and developed by a single person so using the USDP in its entirety would result in overshadowing development with surplus management related work hence only certain aspects of the USDP are relevant to this project.

3.4 Chosen development methodology

Sections 3.2 and 3.3 show that neither approach is ideal for the development of this project; however, the USDP approach improves on the waterfall model by splitting the problem down using components and iterations. The methodology used for this project incorporates areas of the USDP approach with ideas relevant only to this project to reduce management overhead whilst still maintaining tried and tested approach to software development which are seen as “best practices” [14]. Whilst use cases will not be used, UML class diagrams will be used to describe interaction between system components and the basic functionality of certain classes. Figure 3.1 illustrates the proposed methodology. The USDP activities omitted in Figure 3.1 include the initial requirements capture phase for each component, instead defining the requirements at the start of the project. This is made possible since the requirements are very simple. The testing phase is reduced in scale since it is not critical that the system runs completely bug free and there is minimal margin for user input mistakes. The testing conducted will be aimed to eliminate flaws in the concepts behind the system design, for

example, to ensure that the logical model is fast enough for the solution algorithm. Testing is also conducted for each component which will be automated using multiple random test cases where possible. If automation is not possible or appropriate, the test cases used will be specifically noted together with justifications of why they are required. As in USDP a decision is made at the end of each development lifecycle whether to continue to the next stage or not. If the decision is made to continue then the requirements will be extended to incorporate the aims of the improved system.

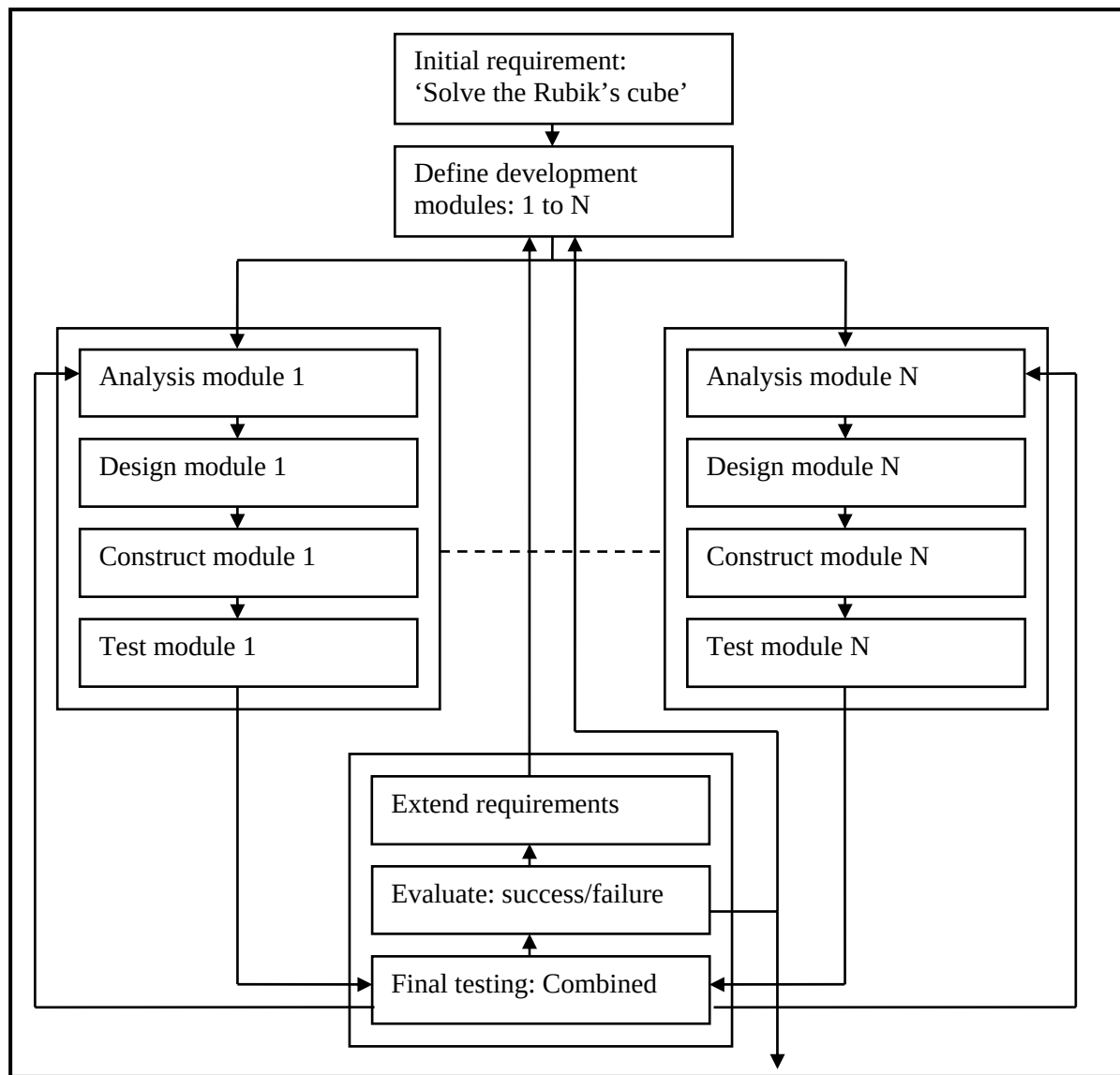


Figure 3.1: Proposed developments lifecycle

3.5 Project plan

The initial plan may be found in appendix C. The project is split into three main sections weighted in complexity according to the project's time constraint. The first section: to design and implement the robot and software to perform simple movements was scheduled for semester 1. This section was intended to be less intensive than sections 2 and 3 due to the Developer's uneven credit split between the two semesters. The second section includes the development of the basic system intended to fulfil the minimum requirements and the development of an improved system intended to fulfil some further aims of the project. The third and final section, representing full completion of the project, is the process of detailing the whole process in the project report and is allocated the full 4 week vacation. All the submitted material has been submitted on time and the first section was carried out according to the plan. However, during the course of the project three notable alterations were made to the plan:

3.5.1 Alteration 1: Extended time required to develop a basic system

Due to the unforeseen difficulty of dealing with large quantities of array values, the time taken to implement the basic requirements was extended by 1.5 weeks (to week 21). This alteration caused a shorter available period of time to implement the advanced system.

3.5.2 Alteration 2: Refining the system

Along with the delayed start time of implementing the basic system and increased non-project, university related dependencies, developing an advanced system required a full re-development of several important aspects. It was found that a greater level of understanding was required here. The consequences forced the project to be further delayed such that development of an advanced system did not finish until week 26. To incorporate these delays into the plan, the decision to combine write up with enhancement implementation was taken. Week 27, originally intended for appendices, references and presentation will now include further work on the evaluation.

3.5.3 Alteration 3: Scrapping the vision system

The decision was taken to implement an improved algorithm first over implementing a vision system because the algorithm was seen to be more relevant in reducing the time required to generate a solution given the initial performance of the system. The complexities of implementing the advanced algorithm, as noted in 3.5.2, resulted in a delay and consequently the vision system idea was scrapped in week 26.

Chapter 4

Solution

4.1 Introduction

The system requires the state or permutation of the cube to be input, a solution to be generated and consequently output in the form of ro-bot instructions that manipulate the cube into its solved permutation. The system can be split into 7 interoperable components:

1. **Logical Cube:** Represents the cube's permutation before and after face rotations are carried out.
2. **Solver:** Takes a logical cube as an argument and builds a list of moves that when applied to the cube will result in the solved permutation.
3. **Interface:** enables the translation from a real cube permutation to a logical cube permutation.
4. **RCX 1:** Receives commands and conducts rotational movements and sensing
5. **RCX 2:** Receives commands and conducts opening and closing movements and sensing
6. **Translation service:** Translates a high level move list (U, B2 etc.) to a low level move list (Rotate jaw clockwise 90° etc.)
7. **Communication:** Sends a low level robot command to the necessary RCX and reports the resulting success or failure.

Chapter 4 provides an overview of the development process used to develop each of the required components in a basic system. Certain components will then be chosen for improvement in a second phase of development.

4.2 General system design

To enable system extensibility and to comply with the Developer's skills, according to the project methodology, an object orientated approach will be used to develop the system. Java provides a useful type of class, an 'interface': Objects that implement the interface are forced to contain the methods specified in the interface e.g. a class implementing the 'ActionListener' interface is a "kind of ActionListener" [20]. Classes can then work with the interface rather than the class itself enabling the actual object passed to the class to be swapped without any further coding. Figure 4.1 illustrates the basic architecture of the system containing 3 interfaces: Any solver object must implement the solver interface thus must include the method 'solve' thus forces the uniformity of all implementation of solver. The two logical cube interfaces are designed to incorporate the requirements of the two types

of solvers discussed in 2.8: ‘Logical Cube’ contains the method ‘doMove’ that may be passed a byte representing a particular face rotation. The ‘Two Phase Logical Cube’ allows different attributes of the cube to be updated in each phase depending on the implementation. Once a solution has been found, the resulting ‘move list’ is passed to the translation service which, in turn, invokes ‘RCXCom’, the class that controls the communications between pc and RCX. The RCX class contains methods to receive a message from ‘RCXCom’ and provides an empty method to convert a message to a robot command. RCX implementations will need to extend this class and override the ‘doMove()’ method to receive messages.

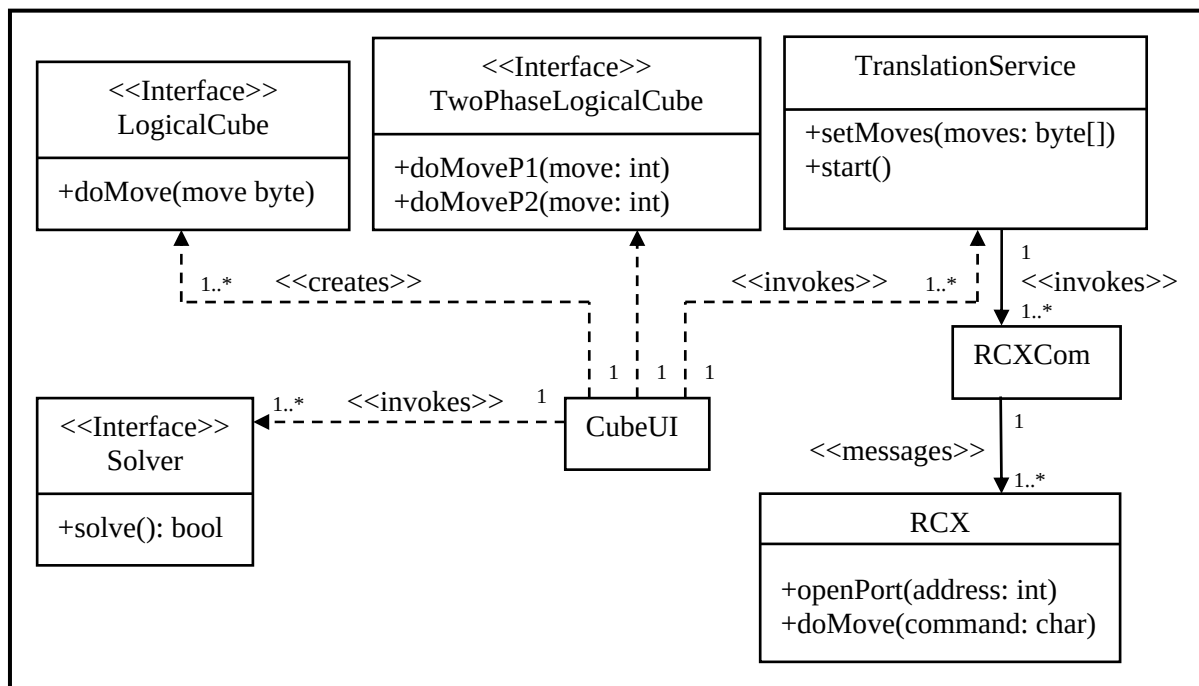


Figure 4.1: UML class diagram showing the 7 system modules

Figure 4.1 is intended as an outline for the system. During the development of the system each of these classes or interfaces will be added to and modified once the full extent of the required interaction between components becomes clear.

4.3 Basic system

4.3.1 Introduction

The basic system is a full implementation of the system using simple techniques to develop each component. The ideas behind the system are based on those used in several similar Rubik's cube solving systems such as that developed by Eric Dietz [17]. The general concept is to mimic a commonly known method for solving the cube manually: The seven stage method (2.8.1).

4.3.2 Robot

Design

The robot required jaws that can open, close and rotate, thus requiring at least 2 motors and 2 sensors to conduct face rotation: The first considered approach was to use six jaws, one for each possible face rotation; however, this requires 6 RCX units. The second considered design was to use two jaws situated at right angles to one another requiring 2 RCX units. To access all 6 face turns the robot will need to apply extra moves to re-orientate the whole cube such that one of the jaws is able to turn the required face thus decreasing the robots efficiency. The third and chosen design is based upon JP Brown's 'cubeSolver' robot [18]: Three jaws (Two opposite and one at right angle). The three jaw system still only required 2 RCX units since all the motor and sensor ports are used in both RCX units. The chosen method for rotational sensing of the jaws is to use touch sensors mounted 90° apart such that each jaw can turn within the 90° gap. Using touch sensors decreases efficiency due to the restricted rotation angle however provides a high level of accuracy and a simple to implement system. Rotational sensors are used for opening/closing movements. The decreased flexibility, caused by twisting rods under high tension, does not represent a problem because tension in the jaw can be predicted since each time the jaw is closed the same amount of tension is built up. The gearing will be discovered through trial and error by applying different ratios until the jaws gripped sufficiently tightly given the motor's power such that the face being turned turns with the jaw without sliding out of place. Gearing for rotation required enough torque to smoothly rotate each face such that firstly the motor does not stop through excess tension in the system and secondly the friction does not cause the movement to be jerky which may decrease the accuracy of the end position. Here there is a trade-off between power and speed so ratios will be selected to maximise speed whilst still providing enough power as described above.

Implementation

Figure 4.2 details the problems encountered when constructing the robot together with the solutions used to combat the problems.

Problem	Solution
Unable to smoothly turn cube face to preserve accuracy	Lubricate cube adjust gear ratios for rotational motors at the cost of slower rotation times
Jaws do not grip cube surface consequently sliding out of place on face turns thus causing the turn to be inaccurate	Adjust gear ratio in opening/closing mechanism. Glue rubber grips to jaw paddles to increase coefficient of friction between paddle and cube
Lego constraints mean that rotational touch sensors do not activate at exactly horizontal or vertical	Re-mount sensors using a rod in the direction of turn such that the sensor can slide along the rod to calibrate the exact distance of turn
Cogs skip when placed under a large amount of pressure	Introduce strengthening bars to hold each cog tightly against its opposing cog
Large cogs attached to rotating part of jaw collide with touch sensors on rotation	Use two sets of smaller cogs to obtain the same amount of gearing in a smaller space.

Figure 4.2: problems encountered when implementing the robot with solutions

Testing

Testing was used to ensure the robot maintains accuracy through multiple operations. These tests comprised of 2 phases. For phase 1 each possible movement was tried using a special java testing class ('rem.java') enabling specific robot moves to be carried out on command. It was necessary to test each rotational movement with the jaws open and closed to ensure that the movement was carried out to a high enough accuracy and speed. During the second phase, 5 iterations requiring varying lengths of robot move sequence were tested and both the speed and the accuracy of the outcome recorded. See appendix B for the results of this test.

4.3.3 RCX programs

Design

Each RCX unit requires a program that extends the communications functionality provided in 'RCX.java' to carry out the required robot actions. As mentioned in section 2.4.2, RCX memory is very limited so the RCX classes will be limited to the following basic functionality:

Gripping RCX

- To perform opening and closing operations for each of the jaws independently
- To perform opening and closing operations for both the side jaws at the same time
- To calibrate each of the jaws such that it closes and opens to the required value

Rotating RCX

- Rotate each jaw independently 90^0 in both directions
- Rotate the two side jaws at the same time in both directions 90^0

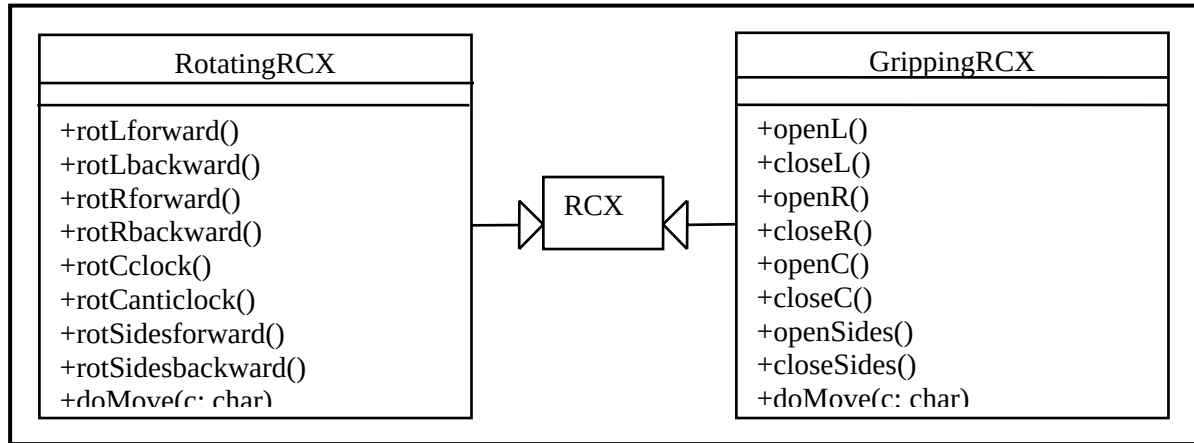


Figure 4.3: UML class diagram showing the details of the RCX classes

Messages that are passed to the 'doMove' method will be in the form of a single character that dictates the movement that is to be conducted. The different jaws are named L: Left, R: Right and C: Centre. The method 'rotSidesforward' in figure 4.3 rotates both L and R forward (L clockwise and R anticlockwise) and the other methods use a similar naming convention. The class diagrams omit the runtime variables since there are two many due to the fact that, as noted in 2.4.2, all variables are initialized at the start of the program to ensure that memory isn't slowly used up during running due to variables being initialized inside loops.

Gripping RCX

Since the gripping mechanism does not require high accuracy when put under stress, rotational sensors provide a good sensing method. Initially the rotational sensor is set to '0': The sensor value when the jaw when it is closed. To open a jaw, the perspective motor is started then a loop reads the value of the rotational sensor until a set value is reached. Once the loop is broken the motor is stopped. For simultaneous jaw movements the motors are stopped inside the loop since the jaws may not become in position at the same time and the loop must run until both jaws are in positions. The system is calibrated by including methods to close each jaw that only halts when the 'VIEW' button is pressed, then setting the sensor value to '0'.

Rotating RCX

The Rotating RCX uses two touch sensors for each jaw connected to one RCX sensor port thus the reading taken from the port represents the setting of both sensors. After a move is carried out 1 of the touch sensors will remain activated. To combat this problem the system does not read sensor values before a set time has passed. This time is set to be the time that the jaw takes to fully clear the sensor. The centre jaw uses a shorter timeout since it rotates more quickly than the side jaws.

Implementation

Gripping RCX

In order to gauge the size of the value generated by opening a jaw to the correct extent it was necessary to adjust the program three times, once for each jaw, such that the value of the rotational sensor is printed to the LCD screen when it changes. Then for each adjustment the jaw can be manually opened to the required extent starting from the fully closed position and the reading taken. This value represents the jaw in its opened position. A button listener then is used to reset the value of the jaws to '0' to enable calibration to take place when the robot is being set up.

Rotating RCX

During implementation the values for the timeout had to be worked out. This was done by simply telling each motor to rotate in each direction until the sensor at the initial point was deactivated and recording the time for this to happen. This time, together with a buffer to ensure that even if the jaw did not move immediately the correct movement is carried out, was used as the timeout: 5 seconds for the side jaws and 1 second for the centre jaw (since it moves more quickly than the sides).

Testing

Testing for the RCX programs was carried out in conjunction with 4.3.3 since the situations investigated in 4.3.3 also test the correctness of the RCX programs.

4.3.4 Communications protocol

Design

The system will use an 'RCXLNPAAddressingPort' [8] which can individually communicate with each of the RCX modules as discussed in 2.5. There is no guarantee that a command will be received so the system requires the following two criteria:

- Acknowledge reception of a valid command
- Report that a move has finished

The host pc sends a command to the RCX; if this is received the RCX will send an acknowledgement byte in return. Once the host pc message has been sent it polls for acknowledgement; if, after a certain amount of time, no acknowledgement is received then the host pc will attempt to retransmit the command. The process is continued until either a finished response is received or an acknowledgement is received. Once these conditions are satisfied the host pc will move on to the next command. The RCX may receive a command and send an acknowledgement that is not received. Since the RCX is busy waiting for the movement to finish the pc continues to resend the command, there is never a need to apply 2 commands of the same one after the other so the RCX program can disregard the resultant duplicate messages. Once the host pc has sent the command and received an acknowledgement it waits for a further message to state that the operation has been completed to check if it is safe to send the next command. The RCX program's main thread is blocked by the loop performing the action until the action is complete. Once this happens the RCX sends the 'finished' message. A timer is used to ensure that, if the 'finished' message is lost in transmission, the host pc simply carries on to the next message after a reasonable delay. This is possible since the RCX is certain to carry out the command if the acknowledgement is sent.

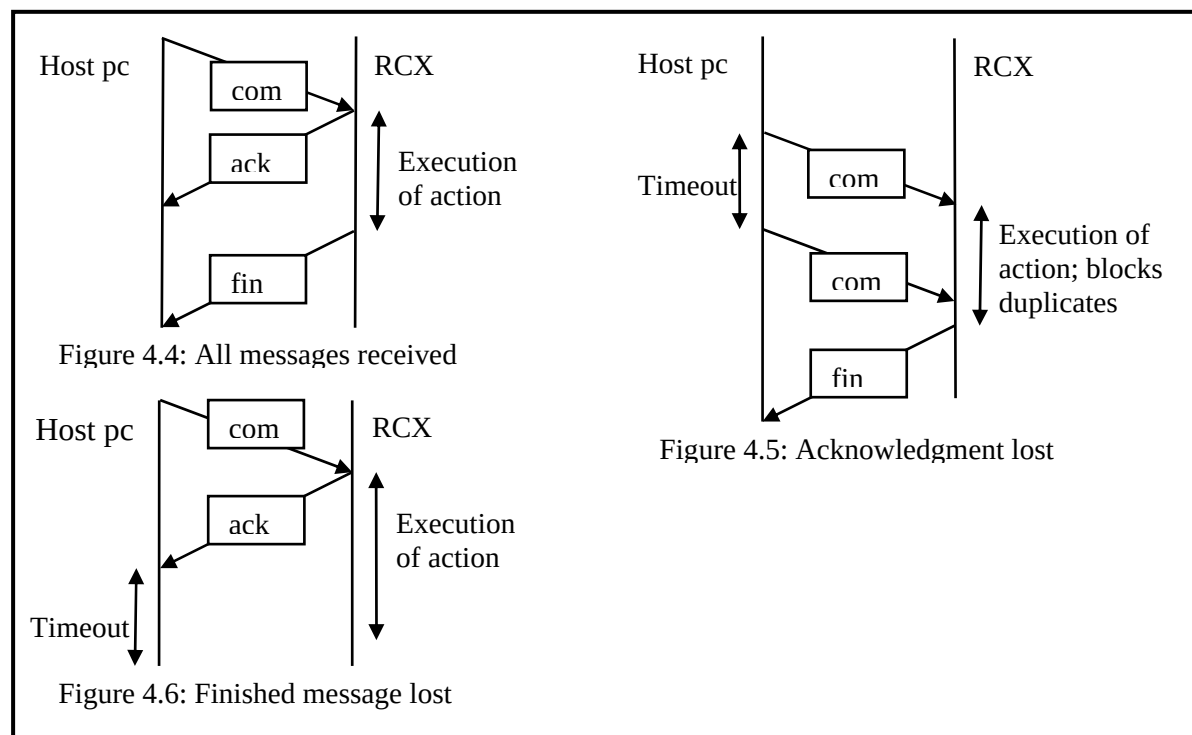


Figure 4.4 shows an instance where all messages (com) and acknowledgements (ack) are received correctly. Figure 4.5 illustrates what happens if an ack is lost in transmission. Figure 4.6 shows what happens when a fin (finished) is lost in transmission. Communication between RCX and pc is carried out between 2 classes: 'RCX.java' for the RCX side and 'RCXCom.java' for the pc side. Figure 4.7 illustrates a UML diagram containing the design of these 2 classes.

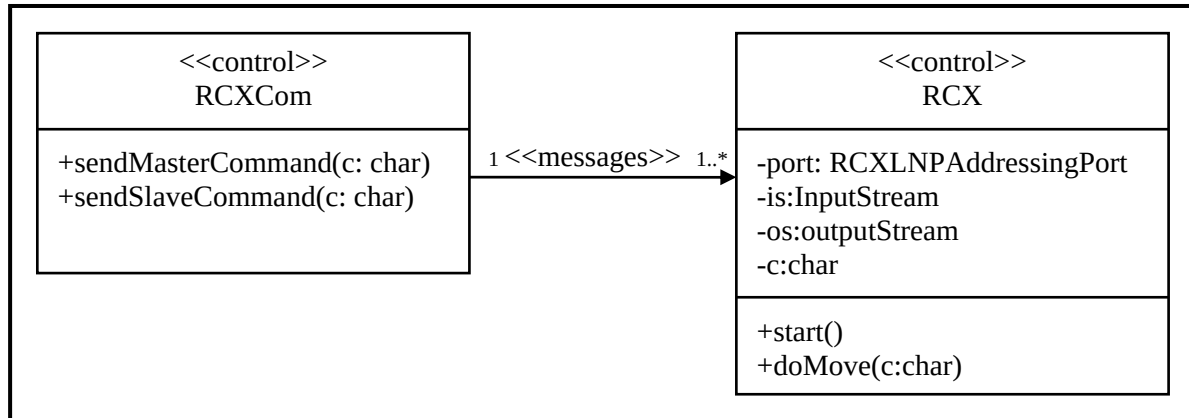


Figure 4.7: UML class diagram showing the communication classes between pc and RCX.

Implementation and testing

The communications protocol was tested by sending sample commands to each of the RCX's, blocking the signal at several points to test the measures implemented from the design for dealing with loss of a message during transmission. The tests were also carried out to ensure messages are always read properly given the problem of reflection and ready time noted in 2.5. The communications protocol is also indirectly tested along with the robot test as part of the combined basic system testing. Although the system does pass all the tests, on several occasions the acknowledgement message was not received correctly. This problem may have caused a system failure if the RCX class did not automatically discard duplicates.

4.3.5 Move translation service

The move translation service translates a simple face turn to a series of robot manoeuvres. Below is a simplified example of this for a 'U' face clockwise rotation:

1. Open centre jaw
2. Rotate both side jaws forward
3. Rotate centre jaw clockwise
4. Close centre jaw
5. Open both sides
6. Rotate centre jaw backward
7. Close both side jaws
8. Rotate right jaw backward
9. Open left jaw and rotate backward and close left jaw

The old 'up' side is now the 'right' side; therefore, new moves are done relative to the current positioning of the cube. To keep track of this, the logical cube object is re-oriented in the same ways as the robot re-orientates the physical cube. The initial colours of each centre sub-cube are recorded allowing the actual face rotation to be carried out on the face containing the centre sub-cube colour matching the initial centre sub-cube colour of the face depicted in the face rotation command.

Implementation and testing

The translation service proved to be difficult to implement since initial attempts commonly included errors in direction of movements and choice of jaws. Debugging the application through testing is a lengthy process due to the time the robot required to conduct each manoeuvre. Several iterations were required before the system implemented each rotation correctly. It became clear that certain extra movements were required to maintain the state of the robot such that the robot jaws would be in a valid place for the next movement. To obtain maximum efficiency all possible robot states would need to be checked resulting in a large increase in code complexity. To guarantee that the system worked, full testing was carried out after the implementation of each unit to ensure that the desired effect was achieved and to ensure that the resulting machine state was valid.

4.3.6 Logical cube

Design

The logical cube represents a cube in logical form by maintaining 6 3x3 byte arrays, each containing the perspective values for the colours found on the perspective cube faces. Appendix D includes a network view of the cube illustrating the means by which sub-faces are transferred to array indices.

Face rotations

Face rotations are carried out using 2 algorithms: Update sub-faces on the face being rotated (figure 4.9) and update sub-faces adjacent to the face being rotated (figure 4.10).

```

Input face: 3x3 Integer array
var out := 3x3 integer array
For(i := 0 to 2) do
    For(j := 0 to 2) do
        out[i][j] = face[2-j][i];
        j = j+1
    od
    i = i+1
od
face = out

```

Figure 4.9: Algorithm for performing a clockwise face rotation

Figure 4.9 rotates, 90° clockwise, all the values in the input array. The anticlockwise rotation algorithm is similar with the alteration: 'out[i][j] = face[j][2-i]' to rotate all values 90° anticlockwise. Together with this, to maintain the rule that edge and corner sub-cube faces must remain constant

since they are fixed, the 12 adjoining faces of the slice that is being rotated need to be updated. Figure 4.10 illustrates an algorithm that conducts this for a rotation of the ‘U’ face.

```

Input: F, B, L, R, U, D: 3x3 Integer arrays representing each face of
the cube.
var a: integer
For(i=0 to 2) do
    a = F[0][i]
    F[0][i] = L[0][i]
    L[0][i] = B[0][i]
    B[0][i] = R[0][i]
    R[0][i] = a
od

```

Figure 4.10: algorithm used to update adjacent faces for a clockwise ‘U’ turn.

Since some faces have different coordinates for sub-faces to other faces, there are individual algorithms to perform each face rotation including slightly different coordinate updates.

Speed of calculation

The time taken to calculate the results of a face rotation may become a key factor when an algorithm that requires the calculation of large numbers of face rotations is used. This time can be reduced, as shown in figure 4.10, by only using 1 temporary value for each array rather than copying the entire array thus minimising the number of calculations required to conduct each face rotation.

Cube rotations

The logical cube contains methods that enable the cube to be re-orientated. These cube rotations are used in the move translation service as described in 4.3.5. The movements are specified as follows:

- **Cube Down:** clockwise rotation about the L R axis
- **Cube Up:** as with cube down but using an anti-clockwise rotation
- **Cube Left:** anticlockwise rotation about the F D axis
- **Cube Right:** same as Cube left but using an anti-clockwise rotation
- **Cube Horizontal flip:** perform 2 sequential left rotations
- **Cube Vertical flip:** perform 2 sequential up rotations followed by a horizontal flip.

The calculation of such cube rotations is carried out by first creating a temporary array for each cube face and secondly replacing the old cube arrays by the temporary arrays corresponding to the faces of the new cube orientation.

Move transposition

If the solver conducts cube rotations during the process of finding the solution, the subsequent face rotations need to be transposed such that the face rotations can be applied without applying the cube rotation when physically solving the cube. Transpositions are simply carried out by using a conditional statement that detects any face rotation and outputs its transposed variant depending on the specified transposition. This specified transposition is simply a cube rotation as specified in 'CubeConstants.java'.

Implementation

Several iterations were used to refine each face rotation such that what was thought to be the correct values were updated. To check the results of a face rotation the method 'printCube' was used to print a network view of cube values which provided a useful debugging tool. The most notable problem encountered was the issue of shallow copying. When copying an object, for example: 'Cube a = b'; Java only copies the pointer of the object. This means that when the object 'Cube a' is updated, the values of 'b' will also be updated as they are basically two references of the same objects. Java only deep copies basic types such as 'int' or 'byte' not 'int[][]' or 'byte[][]'. The benefits of this are to reduce the cost of copying an object and to reduce memory requirements. The problem became evident when attempting to copy the values of each face to a temporary array such that the old values can be swapped with the new values. The solution was to create methods to copy objects such as Cubes or arrays by value rather than by pointer.

Testing

Once the logical cube was in a state that was thought to be correct, further testing was carried out to confirm this. The method 'randomise' applies a sequence of randomly generated moves to the cube and was used to automatically generate test cases. If the number of faces for each colour adds up to 9 after the random moves, then it is likely that the moves applied updated the correct values since a mistake in 1 move would result in at least 1 face being updated incorrectly. The situation in which the faces are updated incorrectly but the number of correct faces maintained would have been noticed in the implementation since many of the resulting face values would be incorrect thus making the print cube output clearly incorrect. Further, more vigorous testing was carried out once the cube solver was created since this program relies on the correct generation of moves on the cube. If the solver could not find a solution then the logical model is incorrect. If after a large number of iterations, the cube solver always finds a solution, the logical cube was seen to be correct.

4.3.7 Solver

Design

For the basic system, the program will solve the cube in the 7 stages as identified in 2.8.1. The general method to solve each stage is to generate a list of face rotations that permute any permutation of the relevant sub-faces to the goal state. Each list of face rotations is then applied to a copy of the current permutation. The shortest move list that generated the resulting correctly permuted copy is then chosen and the ongoing move list is updated with each of the move list's face rotations. This general method provides a simple means to develop the solver and thus makes the 7 stage algorithm a sensible choice. The 5 stage algorithm could equally be implemented using the same technique; however, would require a larger number of possible algorithms given the increased number of relevant possible permutations.

Solving the cross

To solve the cross (place and orientate the top layer edges) the general method is used in four times, once for each edge. In each case the top back edge is corrected. Following each case the cube is rotated using a 'cube left' manoeuvre thus individually solving each top layer edge. Since there are 24 different combinations of the sub-cubes position and orientation for each case, 24 lists of face rotations are used, each permuting distinct edges in distinct orientation to the top back edge. Figure 4.11 illustrates an algorithm to select the shortest list of face rotations. The algorithm takes an array of face rotation indexes (movelist). Each row corresponding to a list of moves and a list of cubes that have had the moves applied to them. The array 'best' is the output and signifies the shortest correct move list.

```

var x := Length[largest element in move list]
for i := 0 to 24 begin
  if U[temp[i]][1][1] = U[temp[i]][0][1] and
    B[temp[i]][1][1] = B[temp[i]][0][1] begin
    if Length[best] != x
      if Length[best] > Length[movelist[i]]
        best = movelist[i]
  else if Length[best] = x
    best = movelist[i]

```

Figure 4.11: Algorithm to output the correct move list to solve the top layer cross

When the algorithm completes, the list of face rotations that generates the correct permutation will be returned. It is important to note that the face rotation lists do not affect the position and orientation of any of the other top layer edges. To do this, if a top layer edge is moved out of place then it must be returned to its place in a subsequent move. This may result in extra moves but ensures that top layer edges remain orientated and in position. For each edge the returned move list is transposed leaving a list of moves that solves the top layer edges without conducting any cube rotations.

Top layer corners

This process is carried out corner by corner similar to that of ‘solving the cross’. Since there are 8 corners each occupying 3 different orientations, 24 combinations of moves lists are required to place and orientate each of the corners one after the other. The move lists are designed to move and/or re-orientate the required corner sub-cube into the bottom layer such that the required ‘up’ side is facing back (unless the corner sub-cube is already correct). Then the move list B, D and BB can be applied to permute the corner into the correct position. Several different move lists are used to permute the required top layer corner to satisfy the above condition from each possible orientation and position. Once this stage is complete, both the top and middle layers are correctly permuted.

Bottom layer cross

Performing a cube vertical flip exposes 1 of 8 possible permutations of bottom layer edge orientation. This figure can be reduced to 4 since the remaining 4 can be accessed by transposing the third by ‘cube left’, ‘cube right’ and ‘cube horizontal flip’. The final permutation can be found by transposing the fourth by ‘cube left’. The first possible permutation has all the edges orientated correctly therefore nothing needs to be done. The second possible permutation has all the edges incorrectly orientated. The following algorithm: F, R, U, RR, UU and FF will move the cube into the third possible permutation in which there are 2 correctly orientated edges adjacent to one another. The move list: F, U, R, UU, RR and FF will re-orientate the remaining edges correctly. The final possible permutation has 2 correctly orientated edges opposite one another. By using the algorithm stated for the second possible permutation all the edge orientations will be solved. Jasmine Lee states in [21] that these are the only possible orientations for the bottom layer edge sub-cubes. To show that this is true we first look at the situation where 3 out of 4 edges are correctly orientated. If this were so then 11 out of 12 edges would be correctly orientated however this contradicts Lemma 1 and so therefore is not possible. The situation in which a single edge is orientated correctly, simply using one of the above algorithms would flip 2 more of the edges thus leading to the situation with 11 out of 12 edges correctly orientated. The cross is then detected using an equivalent method to the top layer cross as stated in stage 1. The chosen moves are then transposed using a vertical flip transposition.

Bottom layer corner placement

The number of incorrectly placed corners in the bottom layer is always either 2 or 4. There must be at least 2 corners incorrectly placed since having a single incorrectly placed corner is clearly impossible. If three corners were incorrectly placed then a single face rotation may be applied to permute the cube such that either 2 or 4 corners are incorrectly placed. Performing an algorithm to swap 2 adjacent edges or some transposed variant suffices once the cube is permuted to have either 2 or 4 incorrectly placed corners. In some situations the corners will be placed opposite one another. In this case the algorithm can be applied three times as shown in Figure 4.12.

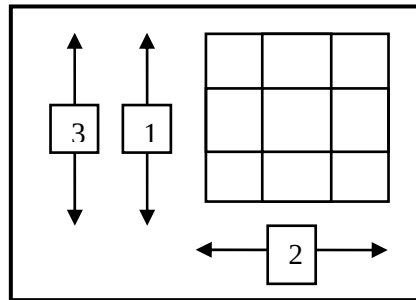


Figure 4.12: Algorithm to re-position diagonal corners correctly

An algorithm can be used to perform a swap of 2 adjacent corners can be used to swap all combinations of corner position. The following move list swaps 2 adjacent corners: L, UU, RR, U, LL, UU, R and U2. This move list can then be transposed to give alternatives for swapping all 4 different combinations of horizontal and vertical swap thus compiling a full list of moves that is guaranteed to correctly position the bottom layer corners. As with ‘Bottom layer cross’ the moves are transposed by ‘cube vertical flip’ then added to the ongoing move list.

Bottom layer corner orientation

Through Lemma 1, the last orientation is forced to be correct if all 3 of the other bottom layer corner orientations are correct. However, unlike edge orientation, since corners have 3 possible orientations the situation exists such that three corners are incorrectly orientated. The algorithm “R, U, RR, U, R, U2, RR the U2” twists three corners clockwise or, performed twice, twists three corners anticlockwise. Hence we aim to obtain a situation such that only 1 corner in the bottom layer is correctly orientated. By Lemma 1 the number of permutations is limited, for example, the permutation with 4 corners twisted in the same direction does not exist. The situation with two incorrectly orientated corners such that twisting them both clockwise leads to a solution does not exist. Further the situation with 4 unorientated corners such that 2 need to be rotated clockwise and 2 need to be rotated anticlockwise similarly does not exist. There are 8 permutations of corner positions: two corners orientated incorrectly that cannot be orientated by the same direction of twist (4 possibilities: 2 adjacent and 2

diagonal), Four incorrectly orientated corners that can be split into 2 pairs (either adjacent or diagonal) that cannot be orientated correctly by the same direction of twist (2 possibilities) and finally 3 corners incorrectly orientated that can be solved by the same direction of twist (2 possibilities: clockwise and anticlockwise). Options 1 to 4 can be brought into the states of options 7 or 8 by twisting 1 corner correctly thus twisting 3 corners out of position. Options 5 and 6 can be solve by twisting 3 corners such that only 1 corner outcomes correctly orientated. Various transposed combinations of twisting 3 corners clockwise or anticlockwise are enough to orientate all corners correctly. Once again the moves are transposed by a vertical cube flip since the cube has been vertically flipped in stage 5.

Bottom layer edge positions

All the orientations are complete, all the corners are in place and now the only task remaining is to place the bottom layer edge sub-cubes without changing the orientations of any other sub-cube. There exists a move list to shift three edge sub-cubes in a clockwise direction. Since there are 4 pieces and 4 positions, there are 12 combinations at this stage; reduced by 12 unreachable combinations through lemma 1 and 2. The 12 missing combinations are those that contain two unsolved edge positions which are opposite or adjacent (6 combinations). The final 6 remaining illegal permutations can be uncovered by applying face rotations to the first 6: all positions need to be placed one to the left or right (2 combinations), two edges need diagonally swapping however two edges need straight swapping (4 combinations). The 12 remaining positions can always be solved by shifting three edges in the same direction leaving one edge un-changed. These combinations consist of one combination where both opposites need swapping, two where diagonals need swapping, one where nothing needs doing and eight (one for each direction) where three 3/4 are misplaced. The algorithm “R2, U, F, BB, R2, FF, B, UU and R2” (swapping the direction of the U turns to switch the direction of rotation) applied to combinations 5 to 12 will result in one of the permutations 1, 2 and 3. The move list (or some transposed version) can then be used to position the bottom layer edges.

Implementation

Each of the seven stages is broken into two methods: The first searches for the best move list; the second performs any required cube rotations and move transpositions calling the first for each stage sub-step. Once a stage is finished then the next stage is called. The process is initiated by calling the solve method implemented from the solver interface. Difficulties arose during implementation since defining the set move patterns used for each stage is a very time consuming process and thus leads to easy mistakes that are difficult to find. The process was made easier by creating a class of constants ‘CubeConstants.java’ that defines constants representing every required cube operation. This enables face rotations to be input as letters rather than numbers.

Testing

Throughout the implementation tests were conducted to reduce error rates at each stage. Once four correct solutions were generated on the finished system, the solver was tested by generating 10000 random cubes, solving each one inside a loop. The resulting permutation was then checked using Cube's 'isSolved' method. If a single permutation was output incorrectly the loop was stopped and an error was raised. The results of the testing showed the solver to provide a legal list of face rotations to solve the cube in each case. The test program was then further developed to output the average number of moves to a solution and the average time taken to generate the solution:

Average number of face rotations:	104
Average time taken:	10 (milliseconds)
Max length of face rotations:	166

Figure 4.13: Basic system: Solver efficiency

Evaluation

The results shown in figure 4.13 indicate that the time taken to generate a solution was very fast however the length of the move list is a limiting factor, particularly since the robot itself takes a large amount of time to conduct even one move. These results indicate that such a system may be used if the limiting factor was computer power rather than the time required to conduct face rotations.

4.3.8 Interface

The first iteration interface uses the manual input technique suggested in 2.9.1. The operator needs to analyse the cube and document its initial state before placing the cube into the robot and starting the solution process. To help this process, a graphical Swing [9] based interface will be used. Six buttons that represent each face colour of the cube need to be presented to the operator. These buttons act synonymously with an artist and a paint palette. 54 'JButtons' are laid out in the shape of a network representing the entire cube. Pressing one of the buttons changes that buttons background colour to the colour of the selected button as shown in figure 4.14. Once all the buttons are pressed the solve button can be pressed to activate the solution. To minimise the cost of using the system, the following further buttons will be added:

‘Solve no robot’

This button generates a solution but does not activate the Robot. The button is useful when testing the functionality of the solver and the logical cube model and may be used if the operator wants to manually solve the cube.

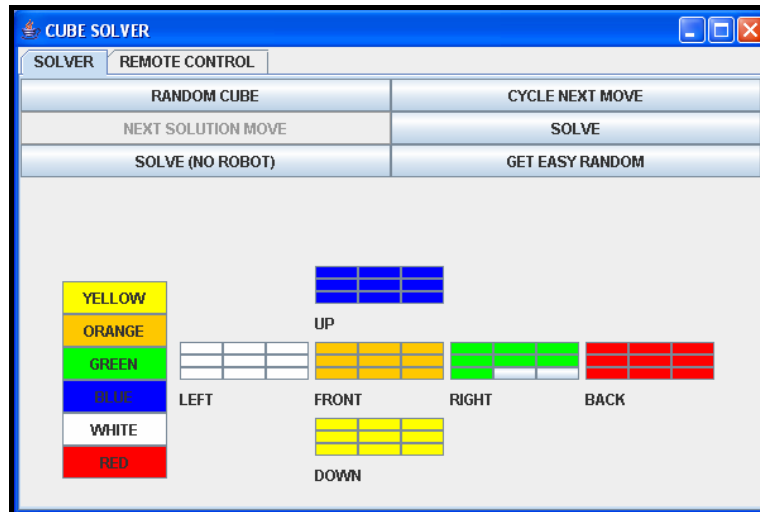


Figure 4.14: Screen shot of operator interface

‘Generate easy random’

This button enables a random solution to be output in less than 10 moves. Solutions are generated for randomised input cubes until one is found that has been solved in less than 10 moves. This is to be implemented to demonstrate the functionality of the robot in a shorter time scale than that of a full solution. Further buttons were added that cycle through each of the generation moves so the operator can put the cube into the state that will be solved in less than 10 moves.

‘Remote control’

The remote control allows the operator to calibrate the opening and closing distance and to carry out various moves for testing and demonstration purposes.

4.3.9 Combined testing

Once all components were developed and tested individually, the system was tested as an integrated unit. The system was tested using the ‘get easy random’ functionality of the interface over five iterations, each time the time and success of the solution is recorded (see appendix B for results).

4.4 Improved system

4.4.1 Introduction

The improved system aims to reduce the average number of face rotations to permute a scrambled cube to the solved state. The following section provides details on the design and implementation of a new logical model and solver based on the two phase method as described in 2.8.3.

4.4.2 Improved logical model

As mentioned in 2.6, the number of computations required to perform a face rotation can be reduced by representing the cube in a different format. The ‘sub-cube’ level permutes ‘sub-cube’ objects by updating the orientation and position of each sub-cube. Due to the low cost of Java shallow copying, references to complicated objects can be re-ordered to represent the permutations with minimal time requirements [29]. Even with the increased speed in performing logical face rotations, the sub-cubed model still takes an unrealistic time when tested in conjunction with a solver such as that described in 2.8.3 (see appendix B for test results). To further increase efficiency, as described in 2.6, the permutation of the cube can be represented by mapping defining characteristics of the permutation to co-ordinates. For example, a number between 0 and 4.3×10^{19} could be used to represent any possible permutation of the cube. Applying a face rotation would change this value. A move table could be constructed containing a column for every possible co-ordinate value and a row for every possible face rotation thus storing the resulting co-ordinate for every coordinate value after each face rotation. Here, each time a face rotation is carried out the resulting co-ordinate value can be looked up in the move table only requiring 1 table lookup. Using 1 table is hindered due to the fact that storing such a table with 4.7×10^{19} values would require an inconceivable amount of memory. The move table can be split according to key aspects that define permutation to reduce the total memory the tables require. In [3] 6 coordinates are used thus reducing the spatial requirements of the corresponding move tables to a reasonable amount. These co-ordinates are designed to re-distribute the search space such that there is a more even split between phase 1 and phase 2, hence evening out the times required to complete each phase. Together with this, only certain co-ordinates that are essential to reach the goal are used to represent the permutation in each phase.

Design

The second iteration consists of 2 further logical cube models. The ‘sub-cubed’ model implements the functionality from both a ‘Logical cube’ and a ‘Two phase logical cube’. This provides compatibility between the single and multiphase logical models and provides the methods required for the setup of the co-ordinate logical model. The sub-cubed model will contain an inner class: ‘subCube’. This simple class stores the orientation (normal, flipped, rotated clockwise etc.) and the name (URF, DL

etc.) of each sub-cube object. Two arrays are maintained containing pointers to the sub-cube objects that are updated according to the changes after a face rotation is carried out. Figure 4.15 illustrates the coordinates, successfully used in [3], required to represent any permutation in the coordinate level.

Phase 1	Phase 2
Corner Orientation (0-2186)	Corner position (0-40319)
Edge Orientation (0-2047)	Edge position (second phase) (0-40319)
UD-Slice (middle layer) (0-494)	UD-Slice (second phase) (0-23)

Figure 4.15: Co-ordinate for two phase algorithm

Corner and edge orientation co-ordinates have already been outlined in 2.6. The UD-slice coordinate takes a value between 0 and 494 and signifies the positions of each of the edges: FR, FL, BL and BR (the middle layer edges). The idea is to place these edges into the middle layer or ‘UD-Slice’ disregarding order. The range of the UD-slice coordinate stems from the number of possible ways to place 4 edges in 12 possible positions disregarding order $((12 \cdot 11 \cdot 10 \cdot 9)/4!)$. Subsequently phase 2 has to solve the order of the 4 middle layer edges (4! combinations), the permutations of the remaining 8 top and bottom layer edges (8! combinations) and the permutations of the 8 corners (8! combinations). Move tables are generated by cycling through each coordinate and decoding that coordinate into a sub-cubed object permutation. Each move is then carried out on the sub-cubed model and the resulting permutations are encoded into the required coordinates and stored in its corresponding move table places. Figure 4.16 illustrates the algorithm used to generate the edge orientation coordinate.

```

SubCubedCube c
int array EOMovTable
for(i := 0 to 17) do                // for each type of move
    for(j := 0 to 2047) do          // for each possible coordinate value
        c := new SubCubedCube();
        c.setEdgeOrientation(j) //decode j to sub-cube model
        c.doMove(i)             // apply move to sub-cube
        EOMovTable[i][j] = c.getEdgeOrientation()
        //encode sub-cube edge orientation to coordinate value
        //then update the move table with value.
    od
od

```

Figure 4.16: Algorithm to generate move table for phase 1 edge orientation

Decoding the coordinates to Sub cubed permutations

To decode such co-ordinates requires varying techniques. Orientations are extracted by dividing the coordinate with decreasing orders of power of either 3 or 2 for corners and edges respectively (from 11 or 7 to 0). The resulting integer multiplied by the power of 3 or 2 is then subtracted from the on going total used in the next case. The value for the last corner or edge can be worked out from the total combined orientation of all the preceding values. To decode positional co-ordinates, the values as shown in the third row of figure 2.2, can be extracted by dividing the total by decreasing factorials (((total/7!)/6!)...). To build a permutation with these values, the positions of each sub-cube needs to be placed in order of decreasing magnitude (figure 2.2) using a system such that the real position of the piece is the position x free places to the left of its magnitude position (where x equals the value extracted by dividing by each factorial).

Encoding the Sub Cubed permutations to coordinates

Once the required move has been carried out, the permutation is then encoded into the required coordinate. The process for orientations is simple and works by multiplying the actual orientation of the coordinate by 3^7 or 3^6 etc... according to the position of that sub cube. Positional information is encoding using a Java adaptation of pseudo code illustrated in [3]. Figure 4.17 shows this algorithm for encoding corner position.

```

var x := 0
SubCube array corners

for(i := 7 downto 1) do
    var s := 0
    for(j := i-1 downto 0) do
        if (corners[j].cornerName() > corners[i].cornerName()) do
            s = s + 1
        od
    od
    x = (x+s)*i
od

```

Figure 4.17: Adaptation of Kociemba's algorithm to encode corner orientation

Move tables in runtime are stored in the form of integer arrays. Since the tables are only used for lookup, the performance of the system can be improved by writing each table to file after its creation. The tables can then be read from the files during the next execution of the program rather than using costly regeneration. The system uses a 'FileWriter' or 'FileReader' object to write or read the move tables to or from memory.

Implementation

The most difficult part of implementing the improved logical model was to correctly generate the move tables. Once this task was completed, the ‘doMove’ methods could be implemented very simply by 1 table lookup for each co-ordinate. Indeed, this implementation could be carried out before the generation of the move tables to reduce the scope for problems to arise in the system.

Testing

To test the improved logical cube, a move list containing 10000 randomly generated face rotations was applied to a sub-face level, a sub-cube level and a coordinate level logical cube. Each of the test logical cubes started in the solved state and the resulting permutations were compared. It was found that all the test cubes matched one another so, since the sub-face level is known to be correct, it is presumed that the sub-cube and coordinate logical models are also correct.

4.4.3 Two Phase Solver

Due to the decreased number of face rotations, on average, required to generate a solution, the two phase solver, as described in 2.8.3, will be implemented; however, the IDA* algorithm used in [3] will not be possible since implementation of the required pruning tables, as described below, is too complicated given the project constraints.

Design

The algorithm to be used applies the depth first iterative deepening [28] in two distinct stages: Firstly an algorithm is run that returns true when the permutation is matched with a ‘reached’ permutation in the pruning table then a secondary search is started using the resulting permutation from the first part that returns true when the phase is satisfied. Each search starts with a movelist of length one, increasing each time the search returns false until the search returns true. Figure 4.18 illustrates Java code for the first part of the algorithm in phase 1 that increments the length of the move list. Figure 4.19 illustrates the search that is performed with each length of move list for part 1. A similar algorithm is conducted to figure 4.18 calling the search algorithm for the second part. This algorithm calls the ‘tryMoveP1’ method instead of ‘tryMatchP1’. Phase 2 algorithms call ‘tryMatchP2’ and ‘tryMoveP2’ instead of ‘tryMatchP1’ and ‘tryMatchP2’ and otherwise perform the same function.

```

int array movelist, TwoPhaseLogicalCube cube, SubCubedCube c

for(i := 1 to 12) do
    if(searchPhase1Part1(movelist[length], cube, i, 0, c)) do
        return true
    od
od

```

Figure 4.18: Algorithm to call a search for increasing sizes of move list

```

searchPhase1Part1(int array movelist, TwoPhaseLogicalCube cube,
                  int length, int index SubSubedCube c)

for(i := 0 to 18) do
    moveList[index] = i
    if(index == length -1) do
        if(cube.tryMatchP1(i)) do //check pruning table match
            cube.doMoveP1(i)
            return true
        od
    od
    else do // if not go to the next depth
        coordinates := cube.getCoordinates();
        cube.doMoveP1(i)
        if(searchPhase1(movelist, cube, length, index+1))
            return true
        else
            cube.setCoordinates(coordinates)
    od
od

```

Figure 4.19: Algorithm to find a move list that generates a permutation in phase 2

The method ‘tryMoveP1’ applies a move exactly as the method ‘doMoveP1’ other than the method does not update the values of the coordinate but simply returns true if the resulting coordinates match those required for the permutation to satisfy phase 1. For each recursion the function is passed the TwoPhaseLogicalCube object with one of the 18 face rotations applied to it. If none of the face rotations succeed the coordinates are reset to their original values to reverse the effects of the move that did not produce the correct results ensuring the cube object remains in the same state before applying each of the face rotations inside the first loop of searchPhase1. The extra value updates add on time to the running of the algorithm through generation of extra moves. The alternative is to create a deep copy of the TwoPhaseLogicalCube object for each recursion of searchPhase1. However, the

cost of generating this object is more than that of generating a single face rotation since creating a deep copy would need to copy all 6 coordinates and the corresponding pruning and move table pointers to the copy. The second phase search is carried out much the same as the first, careful consideration to ensure that the correct `TwoPhaseLogicalCube` object is passed to the start of the algorithm. Since only 3 of the 6 coordinates have been updated in phase 1, the other 3 coordinates need generating. This can be done by applying the moves found in phase 1 to a copy of the original `TwoPhaseLogicalCube` object such that all 6 moves are updated. This copy can then be passed to the Phase 2 algorithm.

Pruning tables

As stated in 2.8.3, phase 1 requires a maximum of 12 face rotations; phase 2 a maximum of 18 rotations. To reduce the time required to generate all potential move lists of these lengths pruning tables are used for each phase. The values stored in the pruning tables represent the permutations that were reached after applying all combinations of face rotations of set length to the solved state. This length for phase 1 is 6 in order to split the search, required to solve phase 1, into two equal parts thus minimising the maximum time required to complete phase 1. Phase 1 requires a 3-dimensional matrix to be built with each of the 3 phase 1 coordinates as an axis such that there is a distinct matrix position for each permutation of the phase 1 coordinates. The problem here is that such a matrix requires $495 \times 2187 \times 2048 = 2,217,093,120$ different values. In [3] the pruning tables are reduced 16 times by using symmetry coordinates; however this results in increased complexity so will not be used in this system. To reduce punning table size only one bit will be stored for each value. The smallest standard unit for a Java basic type is a byte (booleans still take up a byte each) so the table requires 2.06Gb to store the values. Together with this Java uses 16bytes for the creation of each array and a 3-dimentional array creates 1 array for each vertical column, 1 array for each horizontal column containing all the vertical arrays and 1 array containing all the horizontal arrays. To combat these problems, firstly 8 binary values are stuffed into each array value and secondly the array is stored in 1 long list with each column or row for each depth placed end on end thus reducing the setup cost for the array to only 16 bytes. The size if the phase 1 pruning table is now reduced to: $(2048 \times 2187 \times 495) / 8 / 1048576 = 264\text{Mbytes}$. Whilst internally the pruning table is only stored in a single dimension; externally creation, access and mutation of pruning tables values is carried out as if the table where multi-dimensional thus reducing the complexity of implementing the table. Figure 4.10 illustrates the method 'setReached' used to set the correct pruning table value as reached using pseudo multi-dimensional coordinates as parameters.

```

int: width // number horizontal elements
int: height // number of vertical elements
byte array: list //array holding actual values
setReached(int: Hplace, int: Vplace, int: DPlace) begin
    int w:= (width*height*Dplace) + (width*Vplace) + (hPlace)
    int block := list[w/8]
    byte array broken:= decode(block) //get bit window containing value
    broken[mod8(w)] = 1; //set the values as reached
    int outBlock := encode(broken) // get the byte value of the window
    list[w/8] = CastByte(outBlock) // replace pruning table value
end

```

Figure 4.20: Method to set a pruning table value as reached

The phase 2 pruning tables is generated using only 2 of the 3 phase 2 coordinates thus only estimating the length of the required move list. The missing coordinate is the second phase middle layer coordinate. This design decision is forced due to the large space required for the phase 2 pruning table: $(40320*40320*24)/8 + 16 = 4.65\text{Gb}$. Using only 2 coordinates will reduce this figure to 193Mb. Since a match in the pruning table only estimates the minimum number of moves required to reach the goal state, the pruning table search is conducted for move lists of length 7 to minimise the time required to complete the second part of phase 2 since a higher value length, together with extra moves required to reorder the missing coordinate, would take too long to compute.

Storing the pruning table

Performing a depth first search can be a costly process when high depths are required. To reduce the effect of this the tables are stored in persistent memory. This is done by implementing 'java.io.Serializable' in 'PruningTable'. Figure 4.21 illustrates the process of storing a pruning table object in persistent memory.

```

writePruningTableToFile(String: filename, PruningTable p) {
    try{
        ObjectOutputStream out = new ObjectOutputStream(
                                new FileOutputStream(filename));
        out.writeObject(p);
    }catch(FileNotFoundException e){
    }catch(IOException e){}
}

```

Figure 4.21: Java code to write a 'PruningTable' object to file

To read a Pruning table from file a similar process is used in conjunction with an 'ObjectInputStream' and a 'FileInputStream'. The method 'readObject' can be called and the results cast to a PruningTable.

Implementation

The two phase solver was implemented in 2 steps: Initially no pruning table were used so that the algorithm searches for the full length face rotation list in one step. Simple examples were used to aid implementation and debug the program. Once this was done, the second step was to create the 'PruningTable' class that could be used to create both phase 1 and phase 2 pruning tables. The solver could then be advanced to first search for pruning table matches and then search for the desired coordinates.

Testing

These tests incorporate the combined testing phase for the improved system noted in figure 3.1. Appendix B illustrates the times, for each search depth, used in an example taken from the batch test. The test machine uses a 1600Mhz Pentium M processor with 500Mb RAM. Test performances may vary considerably through variations in system configuration, running processes and choice of operating system however the results provided are simply a guide to the expected performance of the system. Clearly 257 days for length 12 in phase one illustrates the need for an informed search method. With the pruning tables, a search for length 12 is reduced to 2 searches for length 6 thus the total time required is $\sim 260 + 1302$ ms rather than 257 days, a considerable improvement. The sample times for phase 2 include an estimated maximum of 555000 ms (~ 9 minutes) for the maximum search depth. The test results also show that the second part of each phase is finished faster than the first for equivalent depths. The reason for this is the method 'tryMatch' used in part 1 requires extra computation to decode the pruning table value into binary data.

To test the reliability, worst case and average performance of the system, automated batch testing was carried out using a sample of 157 randomly generated input cubes (100 random face rotations applied to each). The results showed that a solution was generated in all 157 cases. The worst case performance required 11.76 minutes to be generated. The average time is 2.39 minutes of which 111 seconds is required to load the pruning tables. A system with the capability of holding both pruning tables in memory will result in an estimated reduction to an average of 32.89 seconds to generate a solution. It is possible that by generating the phase 2 pruning table to one less depth, the case requiring 10 face rotations in the second part of phase 2 may not be required. This would result in an increase in time to generate the worst case scenario for the first part of phase 2 which, since phase 1 takes longer than phase 2 depth for depth, does not result in an improvement in performance.

Chapter 5

Evaluation

5.1 Evaluation criteria

The evaluation will be carried out by comparing the system with each of the following criteria:

1. The minimum requirements
2. Non-robotic solution speed
3. The further aims of the project
4. Existing solutions
5. Effectiveness of methodology

The following chapter provides both justifications for the choice of each criterion and the results of evaluating against that criterion.

5.2 Minimum requirements

5.2.1 Justification of minimum requirements criterion

Evaluation against the minimum requirements forms part of the third evaluation criteria suggested in [30] and implemented in [25, 26]. Absolute success when evaluation against the minimum forms the principle criteria to justify the success or failure of the project; therefore it is pivotal to assess each of the minimum requirements for success or failure. Failure to comply with each and every minimum requirement may be caused by high levels of complexity in the minimum requirements or poor performance in system development. If the minimum requirements if prove to be too complex then it is clear that the project is out of scope given the Developer's ability and the project's time constraint.

5.2.2 Evaluation against minimum requirements

Figure 5.1 illustrates each of the minimum requirements together with the result of the comparison with that minimum requirement.

Minimum requirement	Result
Choose a suitable example of a manipulative puzzle problem.	The Rubik's cube was chosen as the manipulative puzzle problem. The cube proved to be both complicated and interesting whilst still remaining simple enough to be worked upon.
Construct a static robot with the physical capabilities to solve the cube	The robot constructed was shown to have the capabilities to solve the Rubik's cube during the project demonstration.
Create an algorithm to generate a series of moves to solve the cube	The algorithm created in iteration 1 of chapter 3 had the capabilities to generate a solution that was shown, through testing, to work.
Choose an appropriate software platform to implement the RCX programs.	The software chosen was the Lejos environment which proved low level enough to provide the required functionality and high level enough to simplify the development process.
Software implementation to solve the cube.	The software implemented in iteration 1 enabled all aspects of the system to function properly.

Figure 5.1: Evaluation against minimum requirements

As all the individual minimum requirements were successfully implemented, the system is deemed to have successfully fulfilled this evaluation criterion.

5.3 Non-robotic solution speed

5.3.1 Justification of non-robotic solution speed criterion

This comparison is not suggested in [25, 26, 30] however aims to answer the general question stated in the project summary: "Can a robot solve a manipulative puzzle faster than a human." Generally, robots are developed to replace actions carried out by humans making an evaluation of human performance to that of a robot an important evaluation criterion for any problem. The Rubik's cube is a puzzle designed to be manipulated by the human hand and so robots are at a significant disadvantage when used to manipulate such objects; therefore, solving a problem robotically faster than solving that problem manually will be significantly more difficult than solving the problem manually in the first place.

5.3.2 Evaluation of non-robotic solution speed to solution speed of the system

The Developer will be used as a base case for testing since the Developer uses a similar system to find a solution as the robot in the basic implementation and the times produced by the Developer are known to be correct enabling a fair comparison to be carried out. The basic system produces a solution requiring an average of 104 moves. The average time required for the robot to complete a face rotation is 1 minute and 8 seconds. A test was carried out (see appendix B) revealing an average of ~3 minutes to input the state of the cube into the system. Therefore the total average time to solution is estimated as 117 minutes for the basic implementation. The Developer's own manual average time to produce a solution, taken from an average of 5 attempts is 3 minutes, considerably lower than the robotic time. The improved system time to solution improves on that of the basic system, producing a solution in an estimated average of 27 minutes, still well above that taken by the Developer. However, the improved system does generate a solution in many fewer face rotations than the Developer requires. The improved system is limited by the speed of the robot and the inefficiency of manually entering the cubes state before solving. A better robot could have been constructed using 6 jaws enabling a face rotation to be mapped to a single robotic manoeuvre. To increase the speed of each robot movement a two gear system could be implemented using two motors, each with different gearing, controlling the same robot part thus enabling extra speed when extra power is not required. However, the inflexibility of using a Lego framework causes unnecessary complexity in robotic design. This complexity could be relieved by using a non-Lego framework. The RCX could still be used in conjunction with a bespoke framework using higher powered motors, directly rotating faces at high speed, connected to the RCX using a relay circuit to control the each motor's operation. A possible root for improving efficiency may be to improve the move translation service to include more robot states thus to minimise the number of non-face rotating robot movements carried out to create a state where the robot can physically rotate the required face. Due to the complexities of implementing such a system it may well be simpler to use 6 jaws, similar to that used in [31], such that the move translation service can directly map face rotations to robotic movements hence reducing cube rotations to zero. The improved system still presents a constraint in the form of the time taken to load the pruning and move tables. This time could be reduced by minimising the pruning table size using equivalence coordinates as describe in [3] thus reducing the size of the pruning tables by a factor of 16 enabling them to be loaded directly into memory at the start of execution. Since the robot is not able to solve the cube faster than the Developer the robot is deemed to have failed this criterion. However, the largely decreased number of moves over the human method shows that a computer may be capable of out-performing a human given an improved robot framework and further developments to the system.

5.3 Further project aims

5.3.1 Justifications of further project aims criterion

The further project aims, specified in point 6 of 1.2, note possible improvements that could be made to the system above and beyond those required simply to make the system work. This comparison forms the second part of the evaluation criteria outlined in [30] and used in [25, 26]. An evaluation of the success or failure of these aims will provide information concerning the complexity of each further aim that is useful for those attempting to solve similar problems. The further aims consist of one main goal: To reduce the time required to solve the cube.

5.3.2 Evaluation against reduced number of moves in generated solution

The improved system reduced the basic system's 104 move average to 24. The basic system required a negligible amount of time to generate a solution; however, due to the nature of the algorithm used in the improved system, this time becomes a defining characteristic of the solution: If the implementation requires more time to generate a solution than the decreased number of moves would save, then such an implementation would not be useful. Testing showed a maximum of 11.76 minutes and an average of 2.39 minutes generated from 157 test cases showing a significant improvement over the basic system. Ideally the system would output the minimum, optimal number of face rotations required to generate a solution; however, in [19] it is stated the extra time taken to generate the solution outweighs the benefits of using an optimal solver. In [3] the two phase solver is tested using 1 million randomly generated test cases to generate, on average, solutions just 2 face rotations more than the optimal solver with a considerable benefit in efficiency. Fulfilment of this criterion is judged upon a reduction in the number of moves required to find a solution showing a considerable decrease in the performance of the basic system and to warrant the extra time taken to generate the solution. Under this assessment, due to the tests detailed above, the improved system has fulfilled this criterion.

5.4 Existing solutions

5.4.1 Justification of existing solutions criterion

Comparison against existing solutions, as used in [15], provides an important perspective on the projects place amongst its competition. Two types of implementation are used for evaluation: The first type implements the two phase algorithm and a vision system to capture the cube starting state. The second system, implementing the seven stage method, will be used to evaluate the basic system. If the basic and improved systems improve upon or match the efficiency of the selected systems, the basic and improved systems may be seen as competitive in the field thus clarifying the quality of the systems produced in this project.

5.4.2 Evaluation against solving software implementing the seven stage method

The chosen system implementing the seven stage method is ‘Rubik’s cube solver’ [17] developed by Eric Dietz. The system finds a solution in 7 stages. Each stage is the same as that used in the basic system with the exceptions that bottom layer edge positions are solved after bottom layer edge orientations rather than after bottom layer corner orientations and bottom layer corner orientations are solved before bottom layer corner positions rather than after. From a sample set of 30 randomised permutations, the test system found a solution in an average of 99 face rotations, 5 less than the 104 face rotation average of the basic system. This inefficiency in the basic system is caused by the limitations in the move lists used to find a solution for each stage. A reduction in the 104 move average could have been made by optimising the output move list by replacing pairs of face rotations with single face rotations where possible; for example, U followed by U may be replaced by U2 and U followed by UU may be removed completely. Further analysis of the basic system may also yield more efficient move lists for each possible permutation for each stage. To clarify the result from the first test a second test was conducted using a set randomly generated permutation for both systems. The test system produced a solution in 85 face rotations and the basic system, a solution in 113 face rotations showing an increase in efficiency of 24% from basic system to the test system. The tests both show a potentially dramatic increase in efficiency from the basic system to the test system hence the basic system has not fulfilled this evaluation criterion. However, the improved systems is guaranteed to yield better results in terms of number face rotations hence making any further improvements to the basic system unnecessary.

5.4.3 Evaluation against solving software implementing two phase method

The chosen test software for evaluating a comparison between two phase solvers is “Cube Explorer” [3]. The comparison will be made between times taken to generate solutions and the number of required output face rotations needed to solve the test cases. The test system was used to generate 30 random permutations yielding an initial average of 24 face rotations to solution. The system required 30 seconds to load tables in to memory and a negligible time to find these solutions. The advanced implementation produces the same solutions to the test systems since the algorithm uses the same two phase system to conduct the search. The considerable difference is found in the time required to generate a solution. Due to the large size of the phase 1 and 2 pruning tables, both tables have to be loaded in separately and discarded during the generation of the solution. This process can take up to 3 minutes per table regardless of the number of moves required to generate the solution. To solve this problem either a machine with extra memory such that both tables can fit in memory at the same time could be used or a reduction in move table size. The test system reduces pruning and move table size by storing the permutation as one of a set of equivalence classes sixteen times smaller than the set of all possible values. The resulting pruning table sizes are 27Mb for phase 1 and 22Mb for phase 2 enabling faster loading times than the improved system. The test system also demonstrates an

improved speed when finding a solution by storing more information in the pruning table. The test system uses 2 bits for each permutation resembling mod 3 (with 1 bit resembling ‘not reached’) of the number of moves to the solution. In this way the pruning table can be used, for phase 1, as an exact metric for selecting the next face rotation such that, when applied, the goal state is one move closer. The test results have demonstrated similar solutions but superior performance from the test system to the improved system. The reasons for this performance deficiency are that of the complexity of implementing a system.

5.5 Effectiveness of methodology

5.5.1 Justification for effectiveness of methodology criterion

Section 5.5.2 includes a critical analysis of the chosen methodology and suggestions where improvements may be made. This criterion was used in [15, 32] and will provide useful information to students undertaking projects using a similar methodology, particularly those mentioned in 6.1.

5.5.2 Evaluation against effectiveness of methodology

As described in 3.5.1, 3.5.2 and 3.5.3 the implementation of the software took more time than was originally planned. These problems were caused by unforeseen, time consuming external factors which should have been catered for in the project methodology. The result of the delays restricted the development of the improved system thus not allowing for a vision system to be implemented. The methodology states a fluid level of completion of the project allowing iterations of development to be started but not completed if the projects time constraints were exceeded thus allowing for the vision system to be disregarded. However, time could have been saved when collecting problem background information if the vision system was never considered thus allowing for further advancements in the solving method. An improved version of the methodology would include a more comprehensive analysis of the project timeline. The time consuming external factors could have been recognised thus allowing for a more accurate version of project requirements. From collection of background information to development, the components represented sub-problems of the task. These components were easily translated into an object-orientated model illustrated by UML class diagrams. This strategy allowed the project to be viewed in terms of several smaller tasks rather than one big task thus providing a successful basis to develop the system. Overall the methodology proved to be successful with the potential for improvement by conducting a more comprehensive analysis of the project timeline.

Chapter 6

Conclusion and future work

6.1 Future work

6.1.1 Vision system

To reduce the need for human input and increase efficiency, a vision system could be used. The colours usually used are yellow, green, orange, blue, red and white. Whilst some colours are easy to distinguish, for example red and white, others are more difficult: white and yellow. If the light cast upon the robot was uniform and controlled then absolute values can be used to record sub-face colours; however, the light's colour or intensity may vary over time or position therefore the distinction between sub-face colours could become more difficult. For use in a real life situation, the vision system would need advancements to deal with these variations, perhaps by making adjustments in hue, saturation and/or intensity and using relative colours.

6.1.2 Advancements in strategy to find a solution

If equivalence coordinates were used in the solving algorithm then the size of the 1-bit pruning table could be reduced by a factor of 16 to 13.4Mb for phase 1 and 10.6Mb for phase 2. This reduction will allow the phase 2 UD-slice coordinate (Figure 4.15) to be included in the phase 2 pruning table whilst still consuming a reasonable amount of memory. Such an inclusion would allow an exact metric when matching a permutation with a pruning table value. The advantage of this is to allow for a more defined estimate on the number of moves required to reach the solved state. With this exact metric the pruning table search could be calculated to a depth of 9 thus splitting the search space in 2 and reducing the maximum time taken for phase 2 by a factor of 10. The second route for advancement would be to reduce the number of phases to 1 phase thus calculating an optimal solution. If a pruning table for all the coordinates is designed requiring a reasonable amount of memory then single phase solutions may be generated quickly. Such a table may contain many empty values since the pruning table search is unlikely to reach, even close, to the total depth required to compute moves for every possible permutation so compression may become a key factor in reducing pruning table size.

6.2 Conclusions

This project has explored the factors involved in a system that has the ability to solve the Rubik's cube. 1.1 presents the problem: Can a robot solve the Rubik's cube faster than a person? Through the development and analysis of two distinct methods: 7 stage algorithm and 2 phase algorithm, an efficient solution has been derived. 5.3 states that the solution still does not outperform a human; however, the concept of the system may be advanced using methods noted in 6.1 in conjunction with a more powerful and accurate robot such that even the fastest human solutions may be outperformed. The project provides a detailed reflection of the complexities involved in implementing communications using Lejos, Logical models representing permutations of the Rubik's cube and algorithms for finding efficient solutions. These reflections are useful for any permutation related computing problem and thus provide an interesting starting point for future projects. Although the project has failed to comply with the evaluation criteria 5.4.2 and does not include an advanced vision system, since the project does comply with the minimum requirements and an advanced system has been successfully implemented, the project has been a success whilst still leaving plenty of scope for advancements and refinements for any future final year project.

Bibliography

- [1] Rubik/Seven Towns Ltd, **Rubik's Cube™**, URL:<http://www.rubiks.com/> [21st April 2007].
- [2] Singmaster, David., (1981), **Notes on Rubik's Magic Cube**, Harmondsworth: Penguin, 5th Edition, pp.3, 4, 12, 19, 25.
- [3] Kociemba, Herbert, **Cube Explorer 4.0**,
URL:<http://www.kociemba.org/cube.htm/> [21st April 2007].
- [4] Lego Group, **Robotics Invention System™**,
URL:<http://mindsotrms.lego.com/> [21st April 2007].
- [5] Ferrari, Giulio et al., (2002), **Programming Lego® Mindstorms™ with Java**, Rockland, MA: Syngress Publishing Inc, pp.11-19.
- [6] Baum, David., **Not Quite C**, URL:<http://bricxcc.sourceforge.net/nqc/> [21st April 2007].
- [7] Noga, Markus., **LegOS**, URL:<http://legos.sourceforge.net/> [21st April 2007].
- [8] Solorzano, Jose et al., **Lego Java Operating System**,
URL:<http://lejos.sourceforge.net/> [21st April 2007].
- [9] Sun Microsystems Inc., **Java**, URL:<http://java.sun.com/>, [21st April 2007].
- [10] Scherphuis, Jaap., (2007), **Computer Puzzling**,
URL:<http://www.goecities.com/jaapsch/puzzles/compcube.htm> [21st April 2007].
- [11] Harris, Dan. & Vandenbergh, Laars., (2007), **VH system for solving the Rubik's cube**,
URL:<http://www.cubestation.co.uk/cs2/index.php?page=3x3x3/3x3x3/> [21st April 2007].
- [12] Hopkins, Johns., (2002), **Adventures in group theory : Rubik's Cube, Merlin's machine, and other mathematical toys Adventures in Group Theory**, London: Johns Hopkins University Press, pp.56, 237-238.

- [13] Russell, Stuart. & Norvig, Peter., (2003), **Artificial Intelligence, A modern Approach**, Great Britain: Prentice Hall, 2nd Edition, pp.94-101.
- [14] Bennett, Simon., McRobb, Steve. & Farmer, Ray., (2005), **Object-Orientated Systems Analysis and Design using UML**, London: McGraw-Hill, 3rd Edition, pp.50, 53, 57, 119.
- [15] Nguyen, Nam, (2006), **Event management system for unim8 limited**, Dissertation (University of Leeds), URL: <http://www.comp.leeds.ac.uk/fyproj/reports/0506/Nguyen.pdf>, [23rd April 2007], pp.19.
- [16] Jacobson, Ivar., Booch, Grady. & Rumbaugh, James., (1998), **The Unified Software Development Process**, Reading, MA: Addison Wesley Longman
- [17] Dietz, Eric., **Rubik's Cube Solver (version .505)**,
URL:<http://www.wrongway.org/cube/solve.html> [21st April 2007].
- [18] Brown, JP., **Cube Solver**, URL:<http://jpbrown.i8.com/cubesolver.html> [21st April 2007].
- [19] Kociemba, Herbert., (27th March 2007), **Email**, kociemba@t-online.de.
- [20] Charatan, Quentin. & Kans, Aaron., (2001), **Java: The first semester**, Berkshire: McGraw-Hill, pp.242.
- [21] Lee, Jasmine., (2007), **Beginners solution to the Rubik's cube**,
URL:http://www.geocities.com/jasmine_ellen/RubiksCubeSolution.html [22nd April 2007].
- [22] Bagnall, Brian., (2002), **Core Lego Mindstorms Programming**, Upper Saddle river NJ: Prentice Hall PTR, pp.19, 25-26.
- [23] Baum, Dave. et al., (2000), **Extreme Mindstorms: An advanced guide to Lego Mindstorms**, Berkeley, CA: Apress, pp.4, 14.
- [24] ACM & IEEE, (2001), **Computing Curricula 2001, Computer Science**,
URL:http://acm.org/education/curric_vols/cc2001.pdf [23rd April 2007], pp.14.
- [25] Barrett, Adam., (2004), **Lego Mindstorms Robotic Football**, Dissertation (University of Leeds)
URL:<http://www.comp.leeds.ac.uk/fyproj/reports/0304/Barrett.pdf.gz> [23rd April 2007], pp.28.

- [26] Dodds, Christopher., (2005), **Sensory Aided Lego Mindstorms Robotics**, Dissertation (University of Leeds), URL:<http://www.comp.leeds.ac.uk/fyproj/reports/0405/DoddsC.pdf> [23rd April 2007], pp.25.
- [27] University of Leeds, School of Computing, (2006), **AI21 Handbook**, URL:<http://www.comp.leeds.ac.uk/ai21/handbook.pdf> [23rd April 2007], pp.41-42.
- [28] Knight, Richard., Rich, Elaine., (1992), **Artificial Intelligence**, London: McGraw-Hill, 2nd Edition, pp.323-324.
- [29] Wikipedia, (2007), **Object copy**, URL:http://en.wikipedia.org/wiki/Object_copy, [23rd April 2007].
- [30] Murphy, Robin., (2000), **Introduction to AI Robotics**, Cambridge, MA: MIT Press.
- [31] Li, Doug., Lovell, Jeff. & Zajac, Mike., (2005), **Rubik's Cube Solver**, Final project (University of Michigan), URL:<http://www.youtube.com/watch?v=LMeRWUt0tkQ> [24th April 2007]
- [32] Burton, Christopher., (2004), **The Design and Development of Educational Software for Disadvantaged Children**, Dissertation (University of Leeds), URL:<http://www.comp.leeds.ac.uk/fyproj/reports/0304/Burton.pdf.gz> [24th April 2007], pp.48.

Appendix A

Personal reflections

Successes

The robot was developed and demonstrated successfully. The final system outperforms the minimum requirements by reducing the number of face rotations required to generate a solution. Initially the Developer thought that implementing a two phase algorithm to find a solution would be out of the scope of the project; however, a simpler approach was devised making the two phase algorithm within reach. Personally the Developer has learnt about many new aspects of computing thus broadening the Developer's knowledge of the subject especially the details of programming with confined resources. Many of the aspects previously covered throughout the Developer's degree programme have been used in this project. Such practical applications of the theories learnt help to build deeper understanding of not only how the aspects work but why they are required. The management of a project of this size has not been undertaken in the past by the Developer and so provides important experience for large scale projects undertaken in the work place as well as lessons learnt in planning and time management. Overall this project has helped to develop a sense of maturity in the computing context.

Failures

Solving the Rubik's cube presents a large number of problems, arguably too many for an undergraduate final year project. Consequently certain areas of the final system lack in development. One such area is the use of computer vision techniques to capture the cubes initial state. This would have provided the robot with important and useful functionality making the system a fully automatic process. Along with this problem, the report has proved difficult to write due to the large quantity of information required to cover all aspects of the development process. Consequently many of the areas may not be covered in as much detail as would be enough to provide full documentation of every aspect of each problem. Along with vision, the robot's capabilities could have been extended with more time or fewer problem areas by firstly advancing the quality and accuracy of the sensing of each movement and secondly improving the speed of each turn. One idea that has not been developed was to construct an algorithm based on minimising the number of robot moves rather than face rotations however the time constraint ensured this was not possible. Overall the project lacks in depth due to the large number of areas involved in solving the Rubik's cube using a robot.

Future students: Choosing a project

From carrying out a dissertation project it has become clear that the choice of the final year project is perhaps the most important decision made throughout an undergraduate university career. If the chosen problem does not have enough depth then it will become unfulfilling and the project report is likely to be very difficult to write. For this project the opposite was the case, the chosen problem was too demanding. If this is the case then a decision should be made in the early stages of the project to narrow the scope of what is to be done. In the case of this project, the scope was not narrowed before development had begun. When the difficulties of implementing certain aspects became clear a decision was made to concentrate on a particular area of the problem for further development. To allow time for this certain proposed aspects were scrapped as documented in 3.5.3. This modification could have been avoided if the problem was more thoroughly assessed at the time the title was chosen.

Future projects: Carrying out the project; planning

This section relates to my design and project planning experiences. With projects relating to Lego Mindstorms there is a tendency to spend much of the initial time attempting to solve the mechanical problem; an interesting and fulfilling task; however, in the context of a computing project, this time would be better spent on researching possible solutions and planning the development process. Time management is also a key factor in the success or failure of a project. A good project will space out the work load throughout the year rather than rushing to do everything at the last minute. This process may not be as easy as it seems due to the variable non-project dependencies, for example: coursework. Although the exact time a coursework will take may vary considerably from year to year, a rough guide estimate can be devised at the start of the project. In section 3.5.1 the project schedule is delayed due to coursework demands. If the schedule had included extra time to compensate for this increased coursework load then more work could have been done prior to the coursework deadlines to ensure that no delay was made.

Future projects: Carrying out the project; Using Lego Mindstorms

I would strongly recommend projects that make use of the Lego Mindstorms framework for the following reasons: Firstly, the RCX is an excellent simple unit which, due to its restricted execution environment, forces the developer to think about what is really going on behind the scenes. During the course of this project I have advanced my programming abilities considerably by learning how to program in restrictive environments. Secondly, the LeJOS system provides an excellent basis to work with. Using LeJOS enables slick incorporation of the RCX classes into a modern development process such as USDP. Along with modelling, LeJOS provides a simple way to access the RCX's functionality without having to know too much about the RCXC whilst still enabling a developer to access low level functionality if this is necessary.

Appendix B

Test results

1: Development phase 1 combined testing: speed and reliability

Test 1 outcome	Completed ok in 8.76 minutes, 7 face rotations
Test 2 outcome	Completed ok in 10.11 minutes, 9 face rotations
Test 3 outcome	Completed ok in 7.67 minutes, 6 face rotations
Test 4 outcome	Completed ok in 8.63 minutes, 7 face rotations
Test 5 outcome	Completed ok in 13.5 minutes, 11 face rotations
Test 6 (fully random) outcome	Completed ok in 135.8 minutes, 120 face rot.

2: Development phase 1 efficiency statistics

Average time to input cube state into system	(5 independent attempts): 3 minutes 12 seconds
Average time to complete 1 face rotation	1 minute 8 seconds
Average number of moves in a solution	104
Average time to solution	117 minutes

3: Move computation speeds for sub-face and sub-cube levels

Length	Time sub-face level (ms)	Time sub-cube level (ms)
4	190	40
5	2924	661
6	46998	10766
7	762627	173389
8	3.5H (estimated)	46.4M (estimated)
9	2.4D (estimated)	12.5H (estimated)
10	39.12D (estimated)	8.5D (estimated)
11	1.76Y (estimated)	138.7D (estimated)
12	54.5Y (estimated)	5.9Y (estimated)

3: Two Phase algorithm test example.

Length	Time without pruning (ms)	Time with pruning (ms)	Phase 2 time (ms)
5	81	70	40
6	1302	1302	421
7	20880	0	3996
8	5.6M	0	3204 (max ~40000)
9	1.5H (estimated)	0	0 (max 39797)
10	24.2H (estimated)	10	0 (max ~350000)
11	16D (estimated)	70	0
12	257.6D (estimated)	260	0
13	-	-	10
14	-	-	60
15	-	-	590
16	-	-	5718
17	-	-	56341
18	-	-	44023 (max ~555000)

4: Development phase 2 batch testing efficiency statistics

Total number test cases	157
Total time taken during test	358.36396 minutes
Average number of moves	24
Max number of moves	28
Average time to generate solution	2.3982 minutes
Max time to generate solution	11.7615 minutes
Average phase 1 pruning table loading time	61 seconds
Average phase 2 pruning table loading time	50 seconds
Average number phase 2 part 1 moves	6.03
Max number phase 2 part 1 moves	8
Average number phase 2 part 2 moves	7.44
Max number phase 2 part 2 moves	10
Average time to generate solution without pruning table loading time (estimate)	32.89 seconds

Appendix C

Initial Plan

Week	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Milestone																												
Submitables																												
Submit aim and minimum requirements																												
Submit mid project report																												
Submit table of contents and draft chapter																												
Submit project report																												
Minimum requirements																												
Choose example of puzzle problem																												
Choose appropriate robot base																												
Research limitation of robot base																												
Research physical characteristics of base																												
Design and construct prototype components																												
Write sample code to test prototype movement																												
Evaluate/ design robot components																												
Design and construct min requirement robot																												
Research appropriate algorithms for puzzle																												
Design and write algorithm																												
Implement algorithm simulation																												
Analyse and choose software platform																												
evaluate hardware communications																												
Implement software on robot and computer																												
Test minimum requirement robot																												
Evaluate success and failure of min robot																												
Enhancements																												
Research image processing an implementation																												
Design and implement vision component																												
Review more efficient algorithm																												
Review robot design/ re-design																												
Test second round robot																												
Review success and failures																												
Environmental condition testing																												
Write up																												
write mid project report																												
write background research chapter																												
Plan and write table of contents																												
write draft chapter																												
write main report																												
formatting and presentation																												
Write appendices and references																												

Week	Date	Week	Date
1	16/10/06	15	22/01/07
2	23/10/06	16	29/01/07
3	30/10/06	17	05/02/07
4	06/11/06	18	12/02/07
5	13/11/06	19	19/02/07
6	20/11/06	20	26/02/07
7	27/11/06	21	05/03/07
8	04/12/06	22	12/03/07
9	11/12/06	23	19/03/07
10	18/12/06	24	26/03/07
11	25/12/06	25	02/04/07
12	01/01/07	26	09/04/07
13	08/01/07	27	16/04/07
14	15/01/07	28	23/04/07

Appendix D

Definitions

Logical cube – abbreviations

URF – up right front corner

ULF – up left front corner

UBL – up back left corner

UBR – up back right corner

DRF – down right front corner

DLF – down left front corner

DBL – down back left corner

DBR – down back right corner

UR – up right edge

UF – up front edge

UL – up left edge

UB – up back edge

DR – down right edge

DF – down front edge

DL – down left edge

DB – down back edge

FR – front right edge (middle layer)

FL – front left edge (middle layer)

BL – back left edge (middle layer)

BR – back right edge

UD-Slice – layer between the up and down cube faces

RL-Slice – layer between the left and right cube faces

FB-Slice – layer between the front and back cube faces

Sub-cube – one of the 26 cubes that, put together, make up the cube itself

Sub-face – One of the 54 coloured faces making up the cube

Logical cube - corner orientation

Corn	L	LL	L2	R	RR	R2	F	FF	F2	B	BB	B2	U	UU	U2	D	DD	D2
URF	0	0	0	2	2	0	1	1	0	0	0	0	0	0	0	0	0	0
ULF	1	1	0	0	0	0	2	2	0	0	0	0	0	0	0	0	0	0
ULB	2	2	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0
UBR	0	0	0	1	1	0	0	0	0	2	2	0	0	0	0	0	0	0
DRF	0	0	0	1	1	0	2	2	0	0	0	0	0	0	0	0	0	0
DLF	2	2	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0
DLB	1	1	0	0	0	0	0	0	0	2	2	0	0	0	0	0	0	0
DRB	0	0	0	2	2	0	0	0	0	1	1	0	0	0	0	0	0	0

Logical cube – edge orientation

Edge	L	LL	L2	R	RR	R2	F	FF	F2	B	BB	B2	U	UU	U2	D	DD	D2
UR	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
UF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
UL	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
UB	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DR	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
DF	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DL	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
DB	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
FR	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
FL	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
BL	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0
BR	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Logical cube - corner position (is replaced by)

Corn	L	LL	L2	R	RR	R2	F	FF	F2
URF	-	-	-	DRF	UBR	DRB	ULF	DRF	DLF
ULF	ULB	DLF	DLB	-	-	-	DLF	URF	DRF
ULB	DLB	ULF	DLF	-	-	-	-	-	-
UBR	-	-	-	URF	DRB	DRF	-	-	-
DRF	-	-	-	DRB	URF	UBR	URF	DLF	ULF
DLF	ULF	DLB	ULD	-	-	-	DRF	ULF	URF
DLB	DLF	ULB	ULF	-	-	-	-	-	-
DRB	-	-	-	UBR	DRF	URF	-	-	-
Corn	B	BB	B2	U	UU	U2	D	DD	D2
URF	-	-	-	UBR	ULF	ULB	-	-	-
ULF	-	-	-	URF	ULB	UBR	-	-	-
ULB	UBR	DLB	DRB	ULF	UBR	URF	-	-	-
UBR	DRB	ULB	DLB	ULB	URF	ULF	-	-	-
DRF	-	-	-	-	-	-	DLF	DRB	DLB
DLF	-	-	-	-	-	-	DLB	DRF	DBR
DLB	ULB	DRB	UBR	-	-	-	DRB	DLF	DRF
DRB	DLB	UBR	ULB	-	-	-	DRF	DLB	DLF

Logical cube – edge position (is replaced by)

Edge	L	LL	L2	R	RR	R2	F	FF	F2	B	BB	B2	U	UU	U2	D	DD	D2
UR	-	-	-	FR	BR	DR	-	-	-	-	-	-	UB	UF	UL	-	-	-
UF	-	-	-	-	-	-	FL	FR	DF	-	-	-	UR	UL	UB	-	-	-
UL	BL	FL	DL	-	-	-	-	-	-	-	-	-	UF	UB	UR	-	-	-
UB	-	-	-	-	-	-	-	-	-	BR	BL	DB	UL	UR	UF	-	-	-
DR	-	-	-	BR	FR	UR	-	-	-	-	-	-	-	-	-	DF	DB	DL
DF	-	-	-	-	-	-	FR	FL	UF	-	-	-	-	-	-	DL	DR	DB
DL	FL	BL	UL	-	-	-	-	-	-	-	-	-	-	-	-	DB	DF	DR
DB	-	-	-	-	-	-	-	-	-	BL	BR	UB	-	-	-	DR	DL	DF
FR	-	-	-	DR	UR	BR	UF	DF	FL	-	-	-	-	-	-	-	-	-
FL	UL	DL	BL	-	-	-	DF	UF	FR	-	-	-	-	-	-	-	-	-
BL	DL	UL	FL	-	-	-	-	-	-	UB	DB	BR	-	-	-	-	-	-
BR	-	-	-	UR	DR	FR	-	-	-	DB	UB	BL	-	-	-	-	-	-

Network view of cube showing the array indices of each sub-face

U F	0 0 0 1 0 2	B L			
	1 0 1 1 1 2				
	2 0 2 1 2 2				
0 0 0 1 0 2	0 0 0 1 0 2	0 0 0 1 0 2	0 0 0 1 0 2	R	
1 0 1 1 1 2	1 0 1 1 1 2	1 0 1 1 1 2	1 0 1 1 1 2		
2 0 2 1 2 2	2 0 2 1 2 2	2 0 2 1 2 2	2 0 2 1 2 2		
D	2 2 1 2 2 0				
	1 2 1 1 1 0				
	0 2 0 1 0 0				